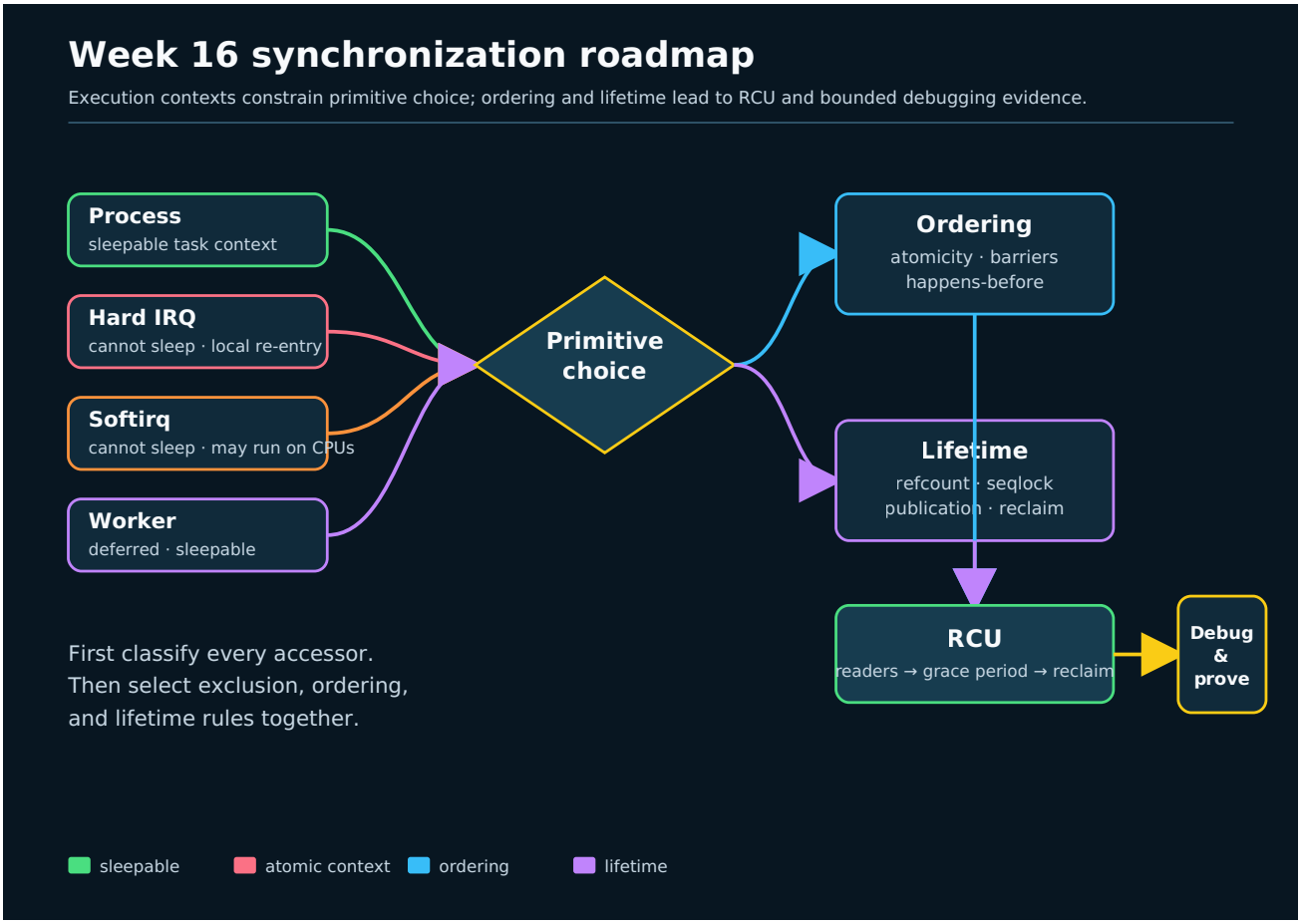
Week 16 — Kernel Synchronization, Memory Ordering, and RCU

Format: seven mechanism-focused lessons, generated theory checks, and bounded userspace laboratories

Readiness state: prepared, not mastery. The learner remains at Week 1 until the course's separate progression process records otherwise.

Week 16 follows one required sequence every day: **syntax → Linux 0.12 → why evolution → exact ToyOS boundary → Linux 6.12/current → bounded evidence**. Syntax establishes how to read the construct. Linux 0.12 supplies a compact historical baseline. The lesson then names the concurrency pressure that forced evolution, states exactly what ToyOS does and does not implement, reads pinned Linux 6.12 before discussing current upstream as version-sensitive, and closes with evidence whose scope and limits are explicit.

The lab is projects/linux-kernel-sync. Its read-only source checker pins `/home/jd/Documents/linux-6.12` to commit `adc218676eef25575469234709c2d87185ca223a`, checks ordered synchronization landmarks, and verifies that the kernel source Makefile hash is unchanged. “Current Linux” means the pinned 6.12 tree unless a later tree is independently identified and checked.



Learning boundary

Linux 0.12 largely relies on one CPU, interrupt exclusion, simple wait queues, and small critical regions; it is a causal starting point, not a modern synchronization recipe. Modern Linux must coordinate SMP CPUs, compiler and hardware reordering, interrupt and preemption contexts, object lifetime, scalable read-mostly access, lock dependency checking, and data-race detection.

ToyOS contributes only its implemented boundary: a single-CPU x86-64 teaching kernel with two ring-0 tasks, timer-driven scheduling, interrupt entry/exit, and small scheduler/console critical regions. It has no SMP memory model, lockdep, KCSAN, production refcount API, seqlock API, per-CPU synchronization framework, RCU grace-period engine, callback offload, PREEMPT_RT, or priority-inheritance mutex subsystem. The userspace labs model selected ideas; they are not kernel implementations and prove no kernel-internal execution path.

Seven-day route

1. Day 1 — Execution Context and Primitive Selection — wrapper, theory check.
2. Day 2 — Atomics, Memory Ordering, and Barriers — wrapper, theory check.
3. Day 3 — Spinlocks, IRQs, and Preemption — wrapper, theory check.
4. Day 4 — Mutexes, rtmutexes, and Sleeping Locks — wrapper, theory check.
5. Day 5 — Refcounts, Seqlocks, and Per-CPU State — wrapper, theory check.
6. Day 6 — RCU Readers, Grace Periods, and Callbacks — wrapper, theory check.
7. Day 7 — Lockdep, KCSAN, RCU Stalls, and Debug Reports — wrapper, theory check.

Evidence and safety contract

Core tests require no root access, load no modules, use no network, and neither mount nor write tracefs. They do not change cgroups, sysctls, CPU state, affinity, scheduler policy, or host-wide tracing configuration. Every concurrent lab uses bounded thread counts and work, arms an alarm, joins all threads, and creates no persistent temporary files. The acquire/release lab checks successful publication and never claims that a forbidden outcome must appear. The priority-inheritance lab observes `PTHREAD_PRIO_INHERIT` support without requesting real-time policy. The epoch lab is clearly a userspace teaching simulator, not Linux RCU.

`make tools` is opt-in. It reports `SKIP` for perf, trace-cmd, bpftrace, and lockstat rather than executing them automatically. A wrapper builds only the project it needs, runs the pinned source check, then runs its bounded day program.

Completion checkpoint

For each day, record the primitive choice, protected invariant, execution context, ordering edge, lifetime rule, evidence observed, and what remains unknown. Passing a wrapper establishes only that the bounded teaching contract ran on this host. It neither establishes mastery nor advances learner state.

Day 1 — Context, Invariants, and Primitive Selection

Target time: approximately 3 hours

> Choose synchronization only after naming the invariant, every context that can touch it, and whether any of those paths may sleep. “Protect this variable” is not enough: the same bytes may require a mutex in process context, an irq-safe spinlock across process and hardirq contexts, per-CPU ownership on a hot path, or RCU when reads dominate and reclamation can wait.

This lesson prepares the mechanism vocabulary ToyOS does not yet implement. Week 11 supplied data races, ownership, mutexes, condition waits, and C atomics. Week 14 supplied hardirq, softirq, threaded-IRQ, worker, preemption, and sleepability contexts. Week 15 supplied task state, wait/wakeup, and runqueue ownership. Today joins those facts into a selection argument.

1. Minimal C and assembly syntax

A lock protects a statement about state, not an address. Write that statement before the lock:

```
/* invariant: count equals the number of linked, live nodes */
mutex_lock(&set->lock);
list_add(&item->link, &set->items);
set->count++;
mutex_unlock(&set->lock);
```

`&set->lock` is the address of one lock object. `->` selects a member through a pointer. Both updates belong to one critical section because exposing only one would violate the invariant. A mutex has ownership: the task that locks it is the task that unlocks it, and contention may sleep.

An atomic read-modify-write covers one atomic object:

```
atomic_inc(&events);
if (atomic_dec_and_test(&refs))
        release_object();
```

It does not automatically protect a neighboring pointer, list, or lifetime. The decrement-to-zero operation is useful only when the reference-count protocol says no new reference can appear after zero.

On x86, these instructions answer different questions:

```
cli                         # mask maskable interrupts on this CPU
sti                         # allow them on this CPU
lock addl $1, events(%rip)  # atomic RMW against other CPUs
```

`cli` does not acquire a shared lock. Another CPU continues to execute. `preempt_disable()` similarly prevents this CPU from scheduling another task through ordinary preemption; it does not exclude another CPU, a hardirq on this CPU, or an NMI. Neither operation by itself states the ordering protocol for shared data. Lock acquire/release operations include ordering needed by their lock contract; Day 2 separates that ordering from exclusion.

2. Linux 0.12: one CPU makes interrupt exclusion look like a lock

Use the local commented *A Heavily Commented Linux Kernel Source Code: Kernel Version 0.12*, Chinese Revision 5, Chapter 8.6, printed pages 351–372, together with `references/EARLY_LINUX_012_MAP.md`. The PDF viewer is about seven pages ahead of the printed page numbers. In `include/asm/system.h`, lines 16–17, the entire interrupt-mask interface is:

```
#define sti() __asm__ ("sti"::)
#define cli() __asm__ ("cli"::)
```

In `kernel/sched.c`, `__sleep_on` lines 179–182 (printed page 359) stores `current` through a wait pointer, changes its task state, and schedules:

```
tmp = *p;
*p = current;
current->state = state;
repeat: schedule();
```

`wake_up`, lines 204–212 (printed page 360), changes the referenced task back to runnable state. Elsewhere, callers surround small shared-state transitions with `cli`/`sti`. On this uniprocessor, masking interrupts prevents an interrupt handler and the interrupted process path from interleaving on that one CPU.

The stable mechanism is the need to preserve an invariant across every context that can modify it, and the separate ability to sleep until an event. The historical details are a single CPU, a single IF bit, fixed task tables, and wait pointers rather than modern wait queues. Early `cli`/`sti` is **not** an SMP lock, an acquire/release pair for another CPU, or a substitute for a memory barrier. The reading map explicitly says Linux 0.12 has no modern SMP locking, C11 atomics, per-CPU runqueues, or RCU.

3. Why the mechanism had to evolve

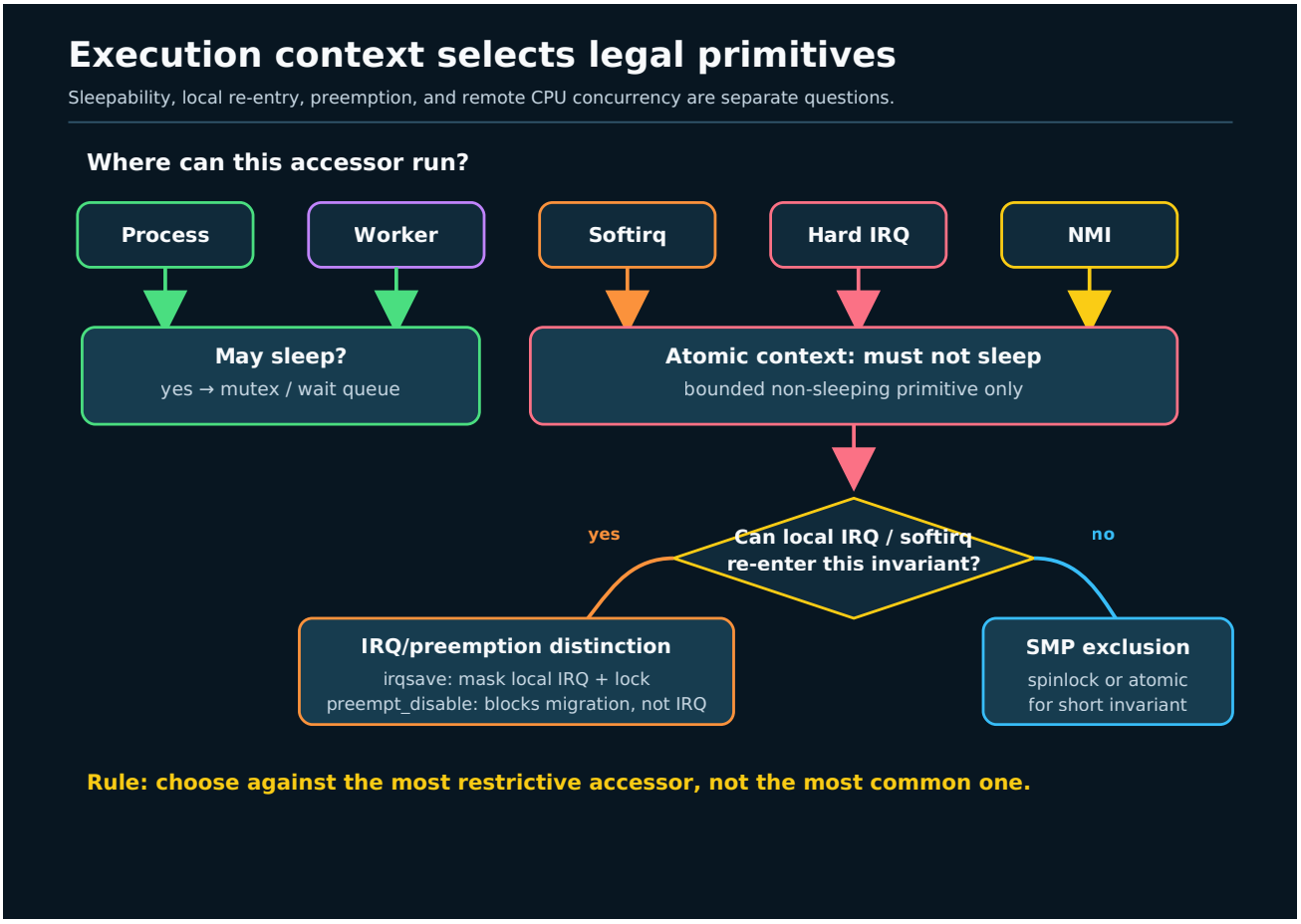
The reason comes before the API.

With two CPUs, local interrupt masking leaves the other CPU free to enter the same critical section. With preemptible kernels, process code may be switched unless the protocol prevents it. With nested hardirq, softirq, and NMI contexts, “interrupts are off” no longer describes every possible entrant. With long waits, spinning wastes a CPU; with hardirq callers, sleeping is impossible. With read-mostly structures, forcing every reader to write one shared lock word destroys scalability. With per-CPU counters, sharing the counter in the first place is avoidable.

Modern selection therefore starts with four questions:

1. **Invariant:** which fields and lifetime transitions must appear indivisible?
2. **Entrants:** process task, hardirq, softirq, worker, NMI, or another CPU?

3. **Waiting:** can the caller sleep, and can contention last long enough that it should?
4. **Shape:** exclusive update, many readers, one-shot event, one scalar, per-CPU ownership, or read-mostly pointer with deferred reclamation?



The context determines whether waiting is legal; the invariant and access shape determine which legal primitive fits.

Disabling preemption answers “may this CPU switch tasks here?” Disabling local IRQs answers “may a maskable hardirq run on this CPU here?” A lock answers “which concurrent entrant owns this invariant?” An ordering primitive answers “which accesses must become visible before which observation?” These answers can be composed, but none implies all the others.

4. The exact ToyOS boundary

The boundary is the checked-in `projects/toyos`, not a future design. `src/tasks.c` has two ring-0 tasks, one `current` pointer, and `TASK_RUNNABLE`, `TASK_RUNNING`, and `TASK_BLOCKED`. `schedule()` mutates task state and calls `context_switch`. IRQ0 enters through `src/isr.S`; vector 32 in `src/arch.c` increments the `volatile uint64_t ticks`, sends the PIC EOI, and calls `scheduler_tick()`. `interrupts_enable()` executes `sti`.

ToyOS is single-core. Its README says it has no locking, and inspection finds no mutex, spinlock, raw spinlock, rwsem, completion, atomic API, per-CPU storage, RCU API, wait queue, softirq, workqueue, or threaded IRQ. The static pipe fields in `src/process.c`, allocator state, scheduler arrays, and `ticks` therefore rely on the current milestone's single-CPU execution and narrow control flow. `volatile ticks` requests observable compiler accesses; it does not make `++ticks` an SMP-atomic increment or publish surrounding state.

The timer handler can switch tasks before the interrupt returns. That is a compact teaching mechanism, not evidence that an arbitrary ToyOS critical section is safe from IRQ re-entry, and not a production rule for switching from hardirq context. `cli` in `qemu_exit()` merely makes the terminal halt loop non-interruptible on that CPU.

The pre-ToyOS design requirement is therefore concrete: before adding a shared subsystem, identify whether its process path can be interrupted by its IRQ path, whether future SMP is in scope, and whether any path waits. Do not retroactively label today's absence of interleaving as a lock implementation.

5. Linux 6.12: selection grounded in implementation

All excerpts below are verbatim from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. Line numbers and behavior are pinned to that checkout.

A mutex: owned and sleepable

`kernel/locking/mutex.c`, `mutex_lock`, lines 281–288

```
void __sched mutex_lock(struct mutex *lock)
{
    might_sleep();

    if (!__mutex_trylock_fast(lock))
        __mutex_lock_slowpath(lock);
}
EXPORT_SYMBOL(mutex_lock);
```

The first semantic signal is `might_sleep()`. Use a mutex in process context when one owner protects an invariant and contention may block. That includes ordinary system-call paths, threaded IRQ handlers, and normal workers, until the caller has entered a non-sleepable region. It excludes hardirq, softirq, NMI, and code holding a spinlock. Week 11's ownership rule still applies: locker and unlocker are the same task.

A spinlock with local interrupt state

`include/linux/spinlock.h`, `spin_lock_irq` and `spin_lock_irqsave`, lines 374–382

```
static __always_inline void spin_lock_irq(spinlock_t *lock)
{
    raw_spin_lock_irq(&lock->rlock);
}

#define spin_lock_irqsave(lock, flags)              \
do {                                \
    raw_spin_lock_irqsave(spinlock_check(lock), flags); \
} while (0)
```

Use the irqsave form when process context and a hardirq on the same CPU can touch the invariant: save the previous mask, prevent that local interrupt from deadlocking on the lock, and acquire mutual exclusion against other CPUs. The flags are restoration state, not lock

ownership. If only process and softirq paths share the object, the corresponding pattern is `spin_lock_bh`. A plain spinlock is enough only when the caller's context contract already handles local re-entry.

`spinlock_t` and `raw_spinlock_t` are not stylistic variants. `raw_spinlock_t` preserves raw spinning semantics, including on PREEMPT_RT, and is reserved for low-level code whose context and latency require it. On RT, ordinary `spinlock_t` semantics may become sleeping rtmutex-based locks. Choosing raw solely because it sounds faster discards RT behavior and increases non-preemptible latency.

A reader/writer semaphore: shared readers that may sleep

`kernel/locking/rwsem.c`, `down_read`, lines 1521–1528

```
void __sched down_read(struct rw_semaphore *sem)
{
    might_sleep();
    rwsem_acquire_read(&sem->dep_map, 0, 0, _RET_IP_);

    LOCK_CONTENDED(sem, __down_read_trylock, __down_read);
}
EXPORT_SYMBOL(down_read);
```

An rwsem fits process-context data with read and write critical sections when read-side sharing is useful and readers may sleep. It is not automatically better than a mutex: reader bookkeeping, cache-line traffic, writer starvation/fairness concerns, and short critical sections can make a mutex better. It is not legal merely because an interrupt handler “only reads”; the read acquisition can sleep.

A completion: one event, not object ownership

`kernel/sched/completion.c`, `complete_all`, lines 67–78

```
void complete_all(struct completion *x)
{
    unsigned long flags;

    lockdep_assert_RT_in_threaded_ctx();

    raw_spin_lock_irqsave(&x->wait.lock, flags);
    x->done = UINT_MAX;
    swake_up_all_locked(&x->wait);
    raw_spin_unlock_irqrestore(&x->wait.lock, flags);
}
EXPORT_SYMBOL(complete_all);
```

A completion expresses “an operation reached this point.” The completing side may signal; the waiting side consumes or observes completion according to the chosen API. It does not protect a data structure like a mutex, and it is not a reusable boolean without a reinitialization/lifetime protocol. Waiting for a completion sleeps, so hardirq code may complete an event but may not wait for it.

Per-CPU state: remove sharing before locking it

`include/linux/percpu-defs.h`, per-CPU operation macros, lines 500–511

```
#define this_cpu_add_return(pcp, val)   __pcpu_size_call_return2(this_cpu_add_return_, pcp, val)
#define this_cpu_xchg(pcp, nval)    __pcpu_size_call_return2(this_cpu_xchg_, pcp, nval)
#define this_cpu_cmpxchg(pcp, oval, nval) \
    __pcpu_size_call_return2(this_cpu_cmpxchg_, pcp, oval, nval)
#define this_cpu_try_cmpxchg(pcp, ovalp, nval) \
    __pcpu_size_call_return2bool(this_cpu_try_cmpxchg_, pcp, ovalp, nval)
#define this_cpu_sub(pcp, val)      this_cpu_add(pcp, -(typeof(pcp))(val))
#define this_cpu_inc(pcp)       this_cpu_add(pcp, 1)
#define this_cpu_dec(pcp)       this_cpu_sub(pcp, 1)
#define this_cpu_sub_return(pcp, val)   this_cpu_add_return(pcp, -(typeof(pcp))(val))
#define this_cpu_inc_return(pcp)    this_cpu_add_return(pcp, 1)
#define this_cpu_dec_return(pcp)    this_cpu_add_return(pcp, -1)
```

Per-CPU storage is an ownership transformation: the common update touches this CPU's instance rather than one globally contended cache line. Aggregation, CPU hotplug, migration, and remote access still require a protocol. A `this_cpu_*` operation is not permission to retain a pointer to one CPU's storage after migration.

RCU: readers do not exclude writers

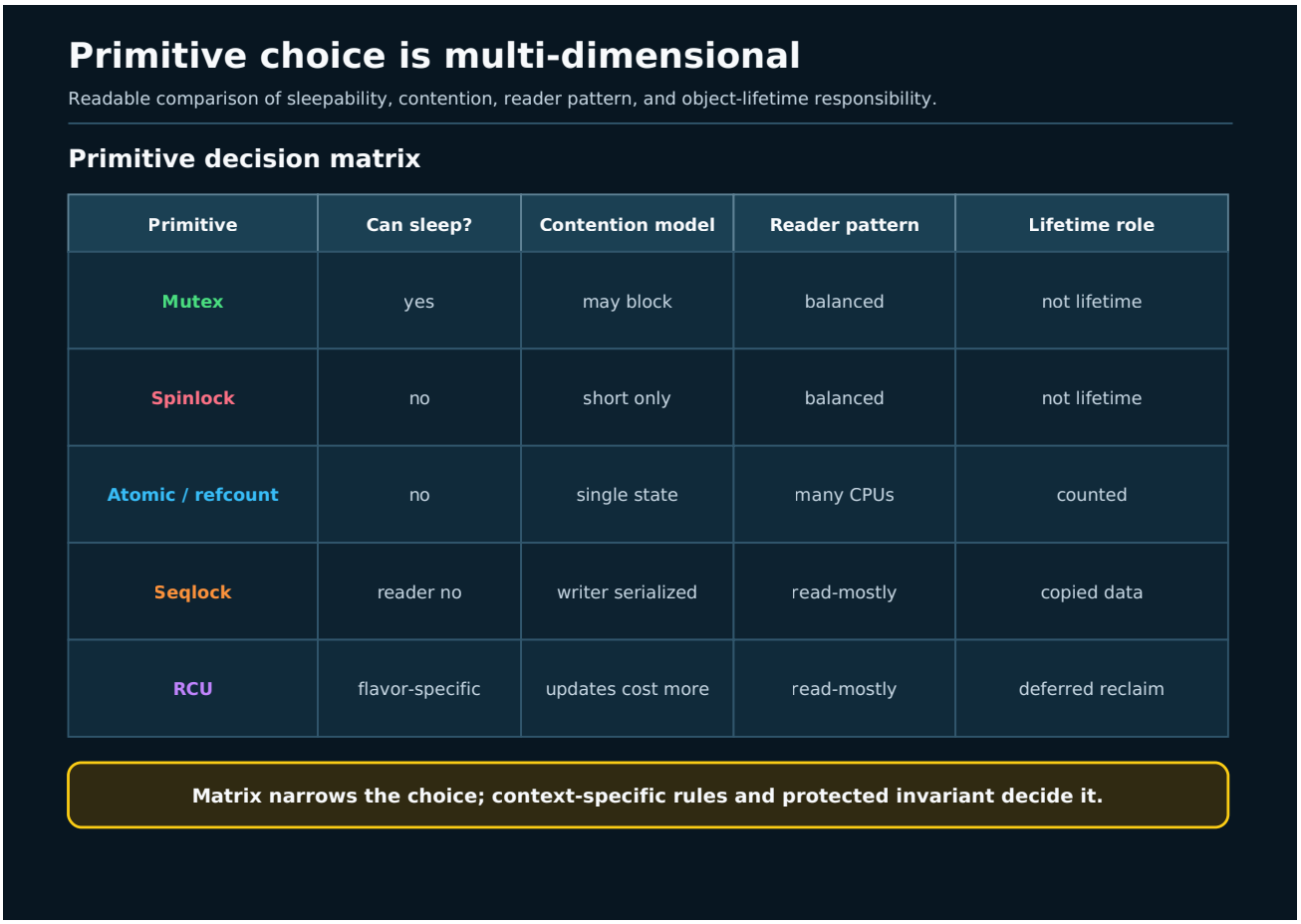
`include/linux/rcupdate.h`, `rcu_read_lock`, lines 845–852

```
static __always_inline void rcu_read_lock(void)
{
    __rcu_read_lock();
    __acquire(RCU);
    rcu_lock_acquire(&rcu_lock_map);
    RCU_LOCKDEP_WARN(!rcu_is_watching(),
             "rcu_read_lock() used illegally while idle");
}
```

RCU fits read-mostly pointer-linked state when readers can tolerate seeing an old or new version and writers can publish a replacement, coordinate with other writers, and postpone reclamation until a grace period. It is a lifetime protocol, not a general replacement for mutual exclusion. Readers do not make a multi-field in-place writer update indivisible. An RCU pointer must be published and dereferenced with the RCU accessors, and freeing the old object must wait. Day 3 will develop that protocol; today the selection limit is enough.

Atomic operations: one state machine, not a tiny lock

An `atomic_t` is suitable for counters, flags, and carefully designed single-object state machines. It is not a mutex compressed into an integer. If correctness requires a list pointer, count, and lifetime to move together, one atomic increment does not provide that transaction. Atomics also do not mean sequential consistency by default in the kernel; ordering depends on the operation. Day 2 supplies that vocabulary.



First eliminate primitives illegal in the caller's context; then choose the smallest protocol that preserves the whole invariant and lifetime.

6. Bounded lab: observation versus source proof

The future lab belongs under `projects/linux-kernel-sync`; this lesson does not claim that project or its output already exists. Keep the exercise bounded and read-only with respect to the host kernel:

1. Build a userspace companion that compares a mutex-protected compound invariant, an atomic counter, and per-thread counters joined at the end.
2. Bound thread count and iterations, and use an alarm or test timeout.
3. Record the compiler, architecture, source revision, and exact command.
4. Run a pinned-source checker that validates the six excerpts above against commit `adc218676eef25575469234709c2d87185ca223a`.
5. Do not load a module, alter IRQ affinity, mount tracefs, or claim entry into hardirq/softirq context.

Observation: a completed host run can show that its mutex invariant held, its atomic increments produced the checked total, and its partitioned counters aggregated to the checked total. Timing may describe that one run.

Source proof: the pinned excerpts establish that `mutex_lock` may sleep, the irqsave wrapper composes local IRQ state with a raw lock, `down_read` may sleep, completion wakeup uses an irq-safe raw lock, per-CPU operations exist, and RCU read locking is instrumented as an RCU critical section in this tree.

Neither proves: that userspace entered kernel interrupt context; that one timing ranking generalizes; that an atomic counter protects compound state; that a booted host matches the pinned tree; or that post-6.12 Linux is unchanged. Unsupported tracing is `SKIP`, not fabricated evidence.

7. Practical selection and limits

Use this reasoning order on a real subsystem:

1. Write the invariant and object-lifetime rule in one sentence.
2. List every entrant and mark process, hardirq, softirq, worker, threaded IRQ, or NMI.
3. For each entrant, mark whether sleeping is legal before and after every lock already held.
4. Ask whether ownership can eliminate sharing: task ownership, per-CPU state, immutable replacement, or a message/completion.
5. If exclusion remains necessary, select a mutex/rwsem for sleepable contention or the appropriate spin/raw-spin variant for atomic context.
6. If one scalar remains, define the atomic state machine and ordering.
7. If reads dominate, define RCU publication, reader, writer-serialization, grace-period, and reclamation rules separately.
8. Check every error, timeout, teardown, and hotplug path against the same invariant.

Common category errors are now explicit:

- `preempt_disable()` is not mutual exclusion against another CPU;
- `local_irq_disable()` is not an SMP lock and does not stop NMI;
- a spinlock's mutual exclusion is not permission to sleep;
- a mutex's sleepability makes it illegal in hardirq and softirq;
- a completion signals progress but does not own protected state;
- an atomic operation covers its object, not an adjacent lifetime;
- per-CPU state postpones aggregation rather than eliminating its protocol;
- RCU protects readers from premature reclamation, not arbitrary in-place writer races.

Week 14 tells you the context. Week 15 tells you what blocking and waking do to task eligibility. Week 11 tells you why an invariant needs a happens-before relation. Primitive selection is the point where all three answers must agree.

Completion prompts

Explain

Explain why a process path sharing a list with a hardirq path needs more than `spin_lock()`: include same-CPU interruption, saved IRQ state, other CPUs, and the no-sleep rule. Then explain why replacing the lock with `atomic_inc` does not protect list reachability and lifetime.

Draw

Draw one object touched by process, hardirq, softirq, and worker contexts. Cross out illegal primitives in each row before selecting one. On a second drawing, partition a hot counter per CPU and mark the separate aggregation protocol.

Observe

For the future bounded lab, preserve only actual stdout, compiler/architecture identity, timeout status, and pinned-source-check result. Label each sentence `host observation`, `Linux 6.12 pinned source`, `ToyOS boundary`, or `hypothesis`.

Source proof

For each excerpt, state the narrow claim it proves and one claim it cannot prove. In particular, explain why the `spin_lock_irqsave` wrapper proves composition of two mechanisms, not that IRQ masking itself is mutual exclusion across CPUs.

Build

Write a primitive-selection note for a hypothetical ToyOS device queue: producer context, consumer context, queue invariant, sleepability, teardown lifetime, primitive choice for single-core today, and the change required for SMP tomorrow. Do not edit ToyOS yet.

Completion requires one final limit statement: **Linux 0.12 `cli`/`sti`, ToyOS's single-core control flow, a userspace host run, and pinned Linux 6.12 source are four different kinds of evidence. None may be silently promoted into another.**

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Context And Primitive Selection

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-001-context-and-primitive-selection.sh
```

- View lab source on GitHub

Day 2 — Atomics, Memory Ordering, and Barriers

Target time: approximately 3 hours

> Atomicity answers whether one operation can tear or lose a competing update. Ordering answers which other accesses must be visible when another CPU observes that operation. A compiler barrier constrains translation. A CPU barrier constrains architectural memory behavior. Correct publication needs the right promise at every layer, not the strongest-looking mnemonic.

Week 11 introduced the C abstract machine and C11 atomics. Week 14 separated interrupt masking from Linux context tracking. Week 15 used ordered task-state publication and wakeup. This lesson bridges those ideas to the Linux Kernel Memory Model (LKMM) before ToyOS grows SMP synchronization.

1. Minimal C and assembly syntax

Portable C atomics name an object, an operation, and an order:

```
#include <stdatomic.h>

_Atomic unsigned ready;
int payload;

payload = 42;
atomic_store_explicit(&ready, 1, memory_order_release);
```

A reader uses an acquire load and reads `payload` only when that load returns the value published by the release:

```
if (atomic_load_explicit(&ready, memory_order_acquire) == 1)
        use(payload);
```

If the acquire reads from the release, the release synchronizes with the acquire. The payload store, sequenced before the release, happens-before the payload load sequenced after the acquire. Atomicity of `ready` alone does not create that edge.

Kernel C uses related but distinct interfaces:

```
WRITE_ONCE(payload, 42);
smp_store_release(&ready, 1);

if (smp_load_acquire(&ready) == 1)
        value = READ_ONCE(payload);
```

This is a conceptual bridge, not an assertion that kernel macros are C11 library atomics. LKMM, kernel compiler conventions, Kconfig, and architecture headers define the kernel operations. The ISO C model defines `atomic_*_explicit`. Do not mix objects or APIs and assume the names make the program portable.

Three assembly forms illustrate the layers:

```
asm volatile("" ::: "memory")  # compiler barrier; emits no CPU fence
mfence                         # x86 CPU fence
dmb ish                        # AArch64 inner-shareable barrier
fence rw,rw                    # RISC-V memory fence
```

The empty inline assembly has a `"memory"` clobber, so the compiler must treat memory as changed across it. It does not ask a CPU to do anything. Conversely, a fence hidden in inline assembly without a compiler memory clobber can fail to constrain compiler motion. Linux barrier macros provide both halves as required by their contract.

2. Linux 0.12: `cli`/`sti` on a uniprocessor, then stop

Read Chapter 8.6, printed pages 351–372, in the local commented *A Heavily Commented Linux Kernel Source Code: Kernel Version 0.12*, Chinese Revision 5, and the Week 11 section of `references/EARLY_LINUX_012_MAP.md`. The PDF viewer is about seven pages ahead of the printed numbering. `include/asm/system.h`, lines 16–17 (the listing identified as Program 14-17 in the course's early-source notes), defines:

```
#define sti() __asm__ ("sti"::)
#define cli() __asm__ ("cli"::)
```

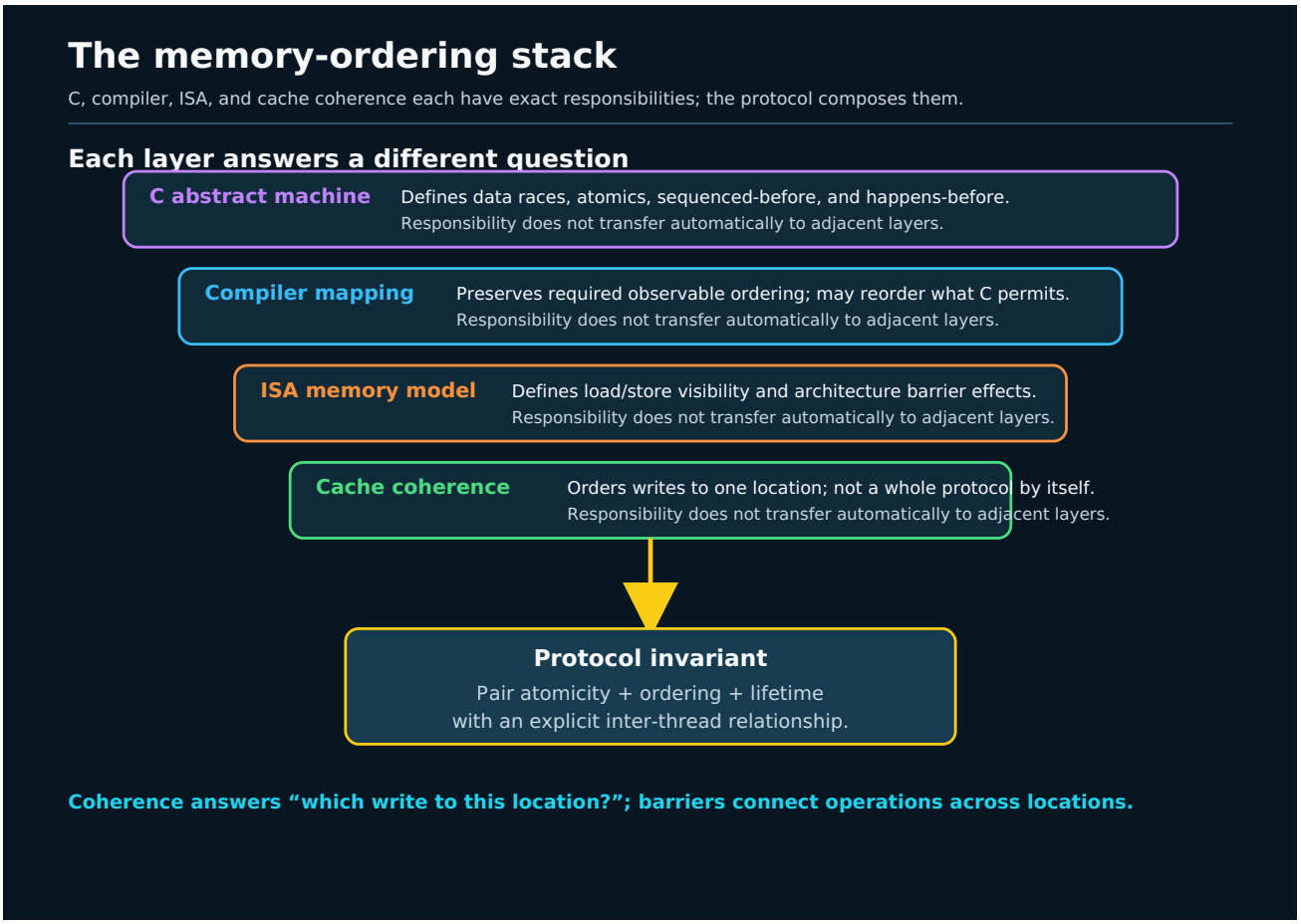
`kernel/sched.c` uses `cli()` at line 233 around the floppy-motor state update and later executes `sti()`. On one CPU, preventing the timer and other maskable interrupts from entering keeps those interrupt paths from interleaving with the update. That is a small implementation of local critical-section control for its machine.

What survives is the need to order and serialize shared-state transitions. What does not generalize is the mechanism. `cli` clears x86 IF on one CPU. It does not stop another CPU, perform an atomic read-modify-write on a shared cache line, or establish a portable C/LKMM publication edge. Linux 0.12 has neither SMP memory-ordering protocols nor C11 atomics. Calling early `cli`/`sti` an SMP lock or an SMP barrier would import a modern property that the source does not implement.

3. Why ordering became explicit

Source order is processed by two independently optimizing machines.

The compiler preserves the language model's observable behavior, not the line-by-line interleaving a programmer imagined. It can keep values in registers, combine accesses, remove redundant loads, and move operations when the model permits. Then the CPU follows its ISA memory model, using out-of-order execution, store buffers, speculation, and coherent caches. Coherence gives an order for writes to one location; it does not create the cross-location publication edge for `payload` and `ready`.



A correct claim names the source-level relation, the compiler constraint, and the ISA mapping. Observing a machine outcome is not a language proof.

SMP exposed those distinctions. A scalar counter needed indivisible read-modify-write operations. A published pointer needed prior object initialization ordered before publication. Lock-free readers needed stable single accesses and lifetime rules. Scheduler wakeup needed task-state stores ordered against the waker's state test. Device registers introduced a different ordering domain from cache-coherent memory.

The resulting vocabulary is intentionally precise:

- `READ_ONCE` / `WRITE_ONCE` constrain compiler access shape and prevent common load/store invention, merging, and tearing for supported scalar sizes;
- relaxed atomics add atomic modification of one object, not cross-object publication;
- release orders earlier operations before publication;
- acquire orders later operations after observation;
- a full barrier orders the relevant accesses on both sides;
- lock acquire/release provides the ordering promised by the lock API;
- dependency ordering is narrow and cannot be inferred from any source-level `if` or pointer expression.

4. The exact ToyOS boundary

The checked-in `projects/toyos` is single-core x86-64. It has no `READ_ONCE`, `WRITE_ONCE`, C `_Atomic`, kernel `atomic_t`, acquire/release helper, SMP barrier, spinlock, RCU accessor, or per-CPU implementation.

`src/arch.c` declares `static volatile uint64_t ticks`; IRQ0 executes `++ticks`, acknowledges the PIC, and calls `scheduler_tick()`. `timer_ticks()` returns `ticks`. `volatile` makes these compiler-visible accesses in this freestanding program. It does not make the increment an atomic RMW against another CPU, and it does not order an unrelated payload. The program's one-CPU interrupt/control-flow assumptions are doing work that the declaration does not express.

`src/isr.S` saves registers, calls `interrupt_handler`, restores registers, and executes `iretq`. `src/tasks.c` switches between two kernel contexts. Neither file contains a second CPU or an inter-CPU observer. The static pipe buffer and indices in `src/process.c` likewise have no SMP publication protocol. They are not a hidden relaxed-atomic implementation.

The exact pre-ToyOS lesson is therefore negative but useful: adding a second CPU would require protocols for scheduler state, allocator metadata, pipe state, device state, and object lifetime before booting that CPU. Adding `volatile` or wrapping those updates in `cli` would not supply those protocols.

5. Linux 6.12: pinned implementation

All excerpts are verbatim from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. Each range is contiguous and pinned to that checkout.

`READ_ONCE` and `WRITE_ONCE`: access shape, not publication

`include/asm-generic/rwonce.h`, `READ_ONCE` and `WRITE_ONCE`, lines 47–62

```
#define READ_ONCE(x)                                    \
({                                      \
    compiletime_assert_rwonce_type(x);                  \
    __READ_ONCE(x);                             \
})

#define __WRITE_ONCE(x, val)                            \
do {                                        \
    *(volatile typeof(x) *)&(x) = (val);                    \
} while (0)

#define WRITE_ONCE(x, val)                          \
do {                                        \
    compiletime_assert_rwonce_type(x);                  \
    __WRITE_ONCE(x, val);                           \
} while (0)
```

The type check constrains supported access sizes; the volatile-qualified access constrains compiler treatment. There is no fence in this excerpt. `WRITE_ONCE(ready, 1)` after a payload store is not, by itself, a release publication on weakly ordered SMP. `READ_ONCE` is similarly not acquire. They are building blocks used inside stronger operations.

The compiler barrier is empty machine code

`include/linux/compiler.h`, `barrier` and `barrier_data`, lines 82–101

```
/* Optimization barrier */
#ifndef barrier
/* The "volatile" is due to gcc bugs */
# define barrier() __asm__ __volatile__("": : :"memory")
#endif

#ifndef barrier_data
/*
 * This version is i.e. to prevent dead stores elimination on @ptr
 * where gcc and llvm may behave differently when otherwise using
 * normal barrier(): while gcc behavior gets along with a normal
 * barrier(), llvm needs an explicit input variable to be assumed
 * clobbered. The issue is as follows: while the inline asm might
 * access any memory it wants, the compiler could have fit all of
 * @ptr into memory registers instead, and since @ptr never escaped
 * from that, it proved that the inline asm wasn't touching any of
 * it. This version works well with both compilers, i.e. we're telling
 * the compiler that the inline asm absolutely may see the contents
 * of @ptr. See also: https://llvm.org/bugs/show_bug.cgi?id=15495
 */
```

The ordinary `barrier()` emits an empty asm template with a memory clobber. The adjacent comment demonstrates why compiler reasoning matters even when no CPU instruction is desired. This primitive cannot flush a store buffer or make another core observe data.

A real release/acquire client

`virt/kvm/dirty_ring.c`, dirty-GFN flag helpers, lines 92–105

```
static inline void kvm_dirty_gfn_set_invalid(struct kvm_dirty_gfn *gfn)
{
    smp_store_release(&gfn->flags, 0);
}

static inline void kvm_dirty_gfn_set_dirtied(struct kvm_dirty_gfn *gfn)
{
    gfn->flags = KVM_DIRTY_GFN_F_DIRTY;
}

static inline bool kvm_dirty_gfn_harvested(struct kvm_dirty_gfn *gfn)
{
    return smp_load_acquire(&gfn->flags) & KVM_DIRTY_GFN_F_RESET;
}
```

This is source evidence that the pinned tree uses release and acquire on the same flag in a real subsystem. It does not, by itself, prove which writer each reader observed; that requires following the callers and the flag state machine. The plain assignment in the middle is also a warning against assigning one order to every access merely because one field is involved.

6. Worked message passing: why the edge exists

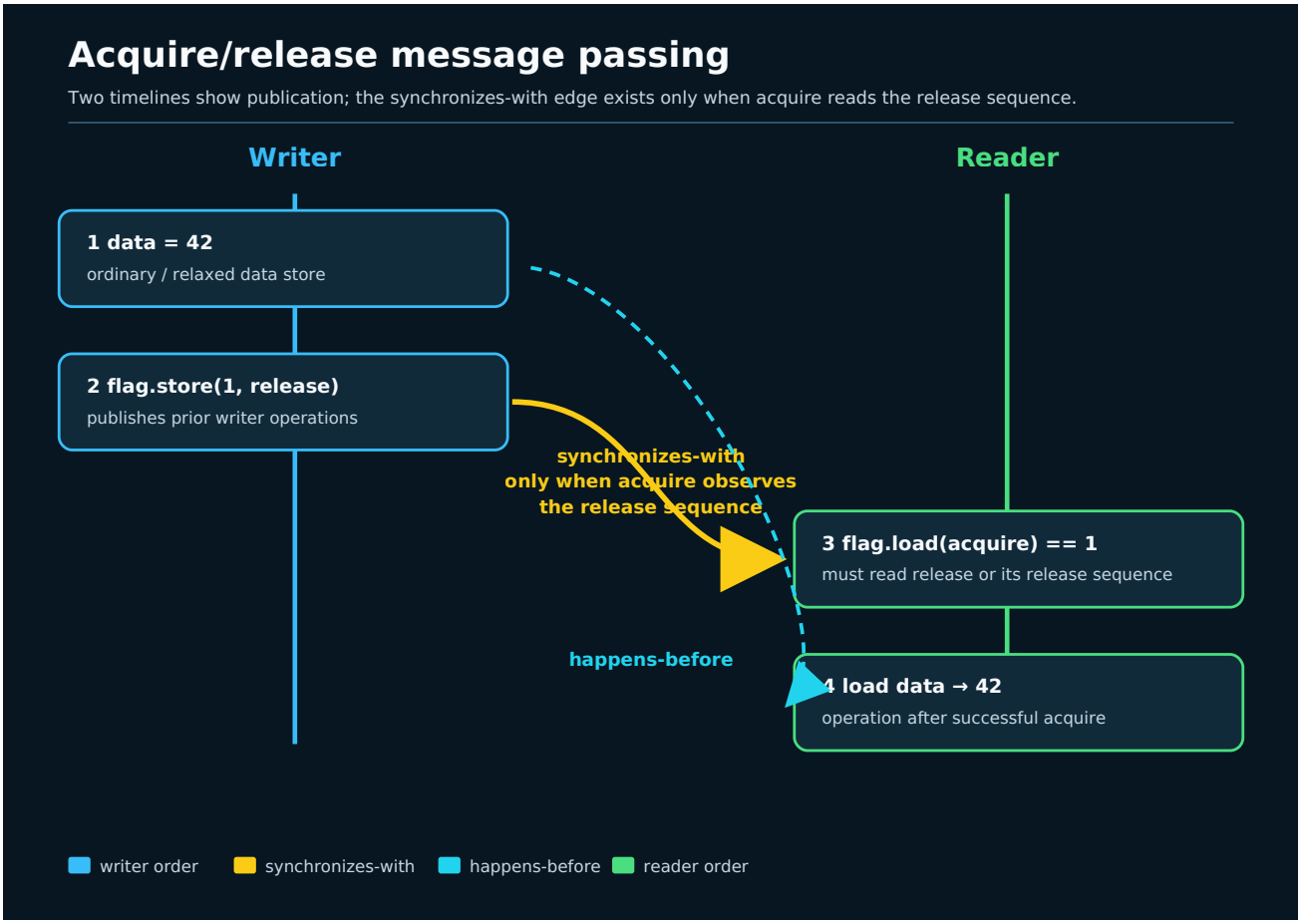
The following is a teaching litmus, not a Linux-source excerpt:

```
initially: payload = 0, ready = 0

CPU 0                                   CPU 1
payload = 42                            r = load_acquire(ready)
store_release(ready, 1)                 if (r == 1)
                                            v = payload
```

Name the relations rather than saying “the barrier flushes everything”:

1. CPU 0's payload store is program-ordered before its release store.
2. CPU 1's acquire load reads the value written by that release.
3. That read-from connection creates the release/acquire synchronization.
4. CPU 1's payload load is program-ordered after the acquire.
5. The transitive chain orders the payload store before the payload load, so this execution reads `42`.



The flag carries the synchronization edge only when the acquire observes the release value. Seeing the old flag creates no permission to read the payload.

Replace both flag operations with relaxed operations and the flag remains atomic, but the cross-object edge disappears. In ISO C, reading the plain `payload` concurrently then creates a data race and undefined behavior. Making `payload` relaxed-atomic avoids undefined behavior but still permits the old payload value. Replace the release/acquire pair with sequentially consistent operations and the contract is stronger than this one-message protocol needs.

A release store orders operations before it; it does not stop later accesses from moving before unrelated later points. An acquire load orders operations after it; it does not retroactively order earlier reader accesses. An acquire-release RMW performs both directional jobs. A full barrier is useful when the protocol needs ordering in both directions without one operation serving solely as the publishing store or observing load.

7. Architecture mappings in this commit

The same kernel-level contract can require different instructions because the ISAs permit different reordering.

x86-64

`arch/x86/include/asm/barrier.h`, SMP barrier and release/acquire, lines 53–72

```
#define __smp_mb()  asm volatile("lock; addl $0,-4(%%" _ASM_SP ")" ::: "memory", "cc")

#define __smp_rmb() dma_rmb()
#define __smp_wmb() barrier()
#define __smp_store_mb(var, value) do { (void)xchg(&var, value); } while (0)

#define __smp_store_release(p, v)                    \
do {                                    \
    compiletime_assert_atomic_type(*p);             \
    barrier();                              \
    WRITE_ONCE(*p, v);                      \
} while (0)

#define __smp_load_acquire(p)                        \
({                                  \
    typeof(*p) ___p1 = READ_ONCE(*p);               \
    compiletime_assert_atomic_type(*p);             \
    barrier();                              \
    ___p1;                                  \
})
```

For normal cacheable memory, x86's ordering permits these release/acquire mappings to use plain accesses plus compiler barriers. The full `__smp_mb` is different: a locked operation supplies the stronger CPU ordering. x86 is not “unreordered”; store buffering still matters, and MMIO has separate rules.

AArch64

`arch/arm64/include/asm/barrier.h`, SMP barriers and beginning of `__smp_store_release`, lines 123–140

```
#define __smp_mb()  dmb(ish)
#define __smp_rmb() dmb(ishld)
#define __smp_wmb() dmb(ishst)

#define __smp_store_release(p, v)                   \
do {                                    \
    typeof(p) __p = (p);                        \
    union { __unqual_scalar_typeof(*p) __val; char __c[1]; } __u =  \
        { .__val = (__force __unqual_scalar_typeof(*p)) (v) };  \
    compiletime_assert_atomic_type(*p);             \
    kasan_check_write(__p, sizeof(*p));             \
    switch (sizeof(*p)) {                       \
    case 1:                             \
        asm volatile ("stlrb %w1, %0"               \
                : "=Q" (*__p)               \
                : "rZ" (*(__u8 *)__u.__c)       \
                : "memory");                \
        break;                          \
```

AArch64 uses `dmb ish` for a full SMP barrier and store-release instructions (`stlrb`, followed by other size cases later in the function) for release. The matching acquire helper below this range uses `ldarb`, `ldarh`, and `ldar`. Weak ordering makes those architectural semantics visible where x86 used plain memory instructions.

RISC-V

`arch/riscv/include/asm/barrier.h`, release/acquire helpers, lines 59–72

```
#define __smp_store_release(p, v)                   \
do {                                    \
    compiletime_assert_atomic_type(*p);             \
    RISCV_FENCE(rw, w);                     \
    WRITE_ONCE(*p, v);                      \
} while (0)

#define __smp_load_acquire(p)                       \
({                                  \
    typeof(*p) ___p1 = READ_ONCE(*p);               \
    compiletime_assert_atomic_type(*p);             \
    RISCV_FENCE(r, rw);                     \
    ___p1;                                  \
})
```

This tree places `fence rw,w` before the release store and `fence r,rw` after the acquire load. Its full SMP barrier, at line 28, is `RISCV_FENCE(rw, rw)`. The sets name predecessor and successor access classes; the release and acquire fences are directional, not aliases for the full barrier.

These headers prove the Linux 6.12 mappings in this checkout. They do not prove how a userspace C compiler lowers `memory_order_release` for arbitrary flags, ABI, tuning, or compiler version. Compile that program for each target and inspect its assembly when that is the claim.

8. Dependencies: narrow tools, not implicit acquire

A **data/address dependency** exists when a loaded value computes the data or address of a later access. The ordering reaches that dependent access, not every later operation. RCU pointer traversal relies on kernel accessors and LKMM rules that preserve the relevant dependency. Copying a pointer through an integer cast, allowing the compiler to prove its value, or using an unapproved transformation can destroy the chain.

A **control dependency** exists when a load determines whether a later store executes. Its scope is the controlled branch. It does not generally order a load before a later load; that shape needs an acquire/read barrier or another documented protocol. Operations after the branches rejoin are not automatically ordered by the earlier condition. Compilers also do not preserve every source `if` as the hardware control dependency imagined by the author.

Therefore:

- do not replace acquire with “there is an `if`”;
- do not treat an address-dependent read as a full barrier;
- use `rcu_dereference()` rather than open-coding dependent pointer loads;
- use documented acquire/release operations when that is the intended edge;
- validate a dependency claim against LKMM documentation and a litmus test, not one disassembly alone.

ISO C's `memory_order_consume` is not a practical bridge here: major compilers commonly strengthen it to acquire, while kernel dependency rules belong to LKMM and kernel accessors.

9. DMA and MMIO: a boundary caveat only

`smp_mb()` orders CPU accesses to normal memory in the SMP domain. Device registers and DMA involve additional actors and architecture-specific APIs. Use `readl`/`writel` and their relaxed variants according to the device's register protocol; use DMA mapping/synchronization APIs according to buffer ownership and coherency. Do not infer that a CPU acquire/release pair flushes a posted MMIO write or transfers DMA-buffer ownership.

ToyOS Week 9 intentionally stops at checked EDU MMIO. Its README states that DMA was not enabled because correct testing also needs pinning, ownership, and device-visible physical-buffer bookkeeping. This lesson preserves that boundary and makes no DMA correctness claim.

10. Bounded lab: observation versus source proof

Align the future exercise with `projects/linux-kernel-sync`; do not claim that the project or any output exists yet. A bounded lab should:

1. run a finite release/acquire message-passing test and a finite relaxed comparison using C atomics;
2. use a timeout and report observed counts without requiring a weak outcome;
3. emit compiler version, target architecture, options, and focused assembly for the publication functions;
4. run pinned-source validation for the six Linux excerpts in this lesson;

5. optionally run an LKMM litmus through existing kernel tooling, recording unavailable tools as `SKIP`;
6. avoid kernel modules, host-wide tracing changes, MMIO, and DMA.

Observation: one host run may show that every release/acquire iteration accepted by the reader carried the expected payload, and may or may not show a weak relaxed outcome. Its assembly proves one compiler's mapping for one target and option set.

Source proof: the excerpts establish the access-shape macros, compiler barrier, one real release/acquire client, and x86-64/AArch64/RISC-V mappings in pinned Linux 6.12.

Not proved: absence of all allowed weak outcomes; equivalence between C11 and LKMM; execution of a shown kernel path by the host; behavior on another architecture; MMIO/DMA ordering; or post-6.12 source stability. Never invent an expected weak-outcome count. Zero is a valid observation and not a proof that the relaxed model forbids the outcome.

11. Practical reasoning and limits

For every lock-free or low-level synchronization protocol, write this proof outline:

1. List each shared object and whether accesses are plain, `READ_ONCE`, atomic, lock-protected, or RCU-protected.
2. Name the invariant and lifetime rule from Day 1.
3. Draw each writer's program order and each reader's program order.
4. Identify the exact read that must observe the exact write.
5. Name the synchronizes-with or LKMM relation that observation creates.
6. Follow transitivity to every payload access the protocol claims to order.
7. State what happens when the reader observes an older value.
8. Check compiler constraints and each supported ISA mapping.
9. Keep normal memory, MMIO, and DMA domains separate.
10. Test with bounded observations and source/litmus proof; do not promote one into the other.

Week 15's wakeup path is a concrete application: waiter state publication and the waker's state test need an order so a wake cannot pass an unpublished sleep. Week 14's `cli`, DAIF, and `sstatus.SIE` manipulate local interrupt masks, not this inter-CPU relation. Week 11's C atomics remain the authority for userspace labs, while LKMM remains the authority for kernel litmus claims.

Completion prompts

Explain

Explain why `WRITE_ONCE(payload, 42); WRITE_ONCE(ready, 1);` does not by itself publish the payload on weakly ordered SMP. Then explain the exact condition under which a release store and acquire load do publish it.

Draw

Draw the worked message-passing execution with program-order, reads-from, synchronizes-with, and happens-before edges. Beside it draw the relaxed version and mark the missing edge. Add the compiler and CPU layers from the ordering-stack visual.

Observe

For the future bounded lab, preserve actual counts even when the weak count is zero. Record compiler, target, options, timeout status, and assembly. Label the result `host observation`, not `memory-model proof`.

Source proof

For all six excerpts, validate the exact path and inclusive line range against commit `adc218676eef25575469234709c2d87185ca223a`. State one narrow claim per excerpt. Explain why x86's plain release store and AArch64's `stlr` can implement the same kernel contract.

Build

Write, but do not yet integrate, a ToyOS publication design for one immutable object pointer. Include initialization, release publication, acquire observation, old-value behavior, object lifetime, and the single-core build's relationship to a future SMP build. Do not use `volatile` as the ordering argument.

Completion requires this final limit: **atomicity is not ordering, `READ_ONCE` is not acquire, interrupt masking is not an SMP barrier, one architecture's assembly is not a portable C proof, and normal-memory barriers do not silently solve MMIO or DMA ownership.**

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Atomics Memory Ordering And Barriers

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

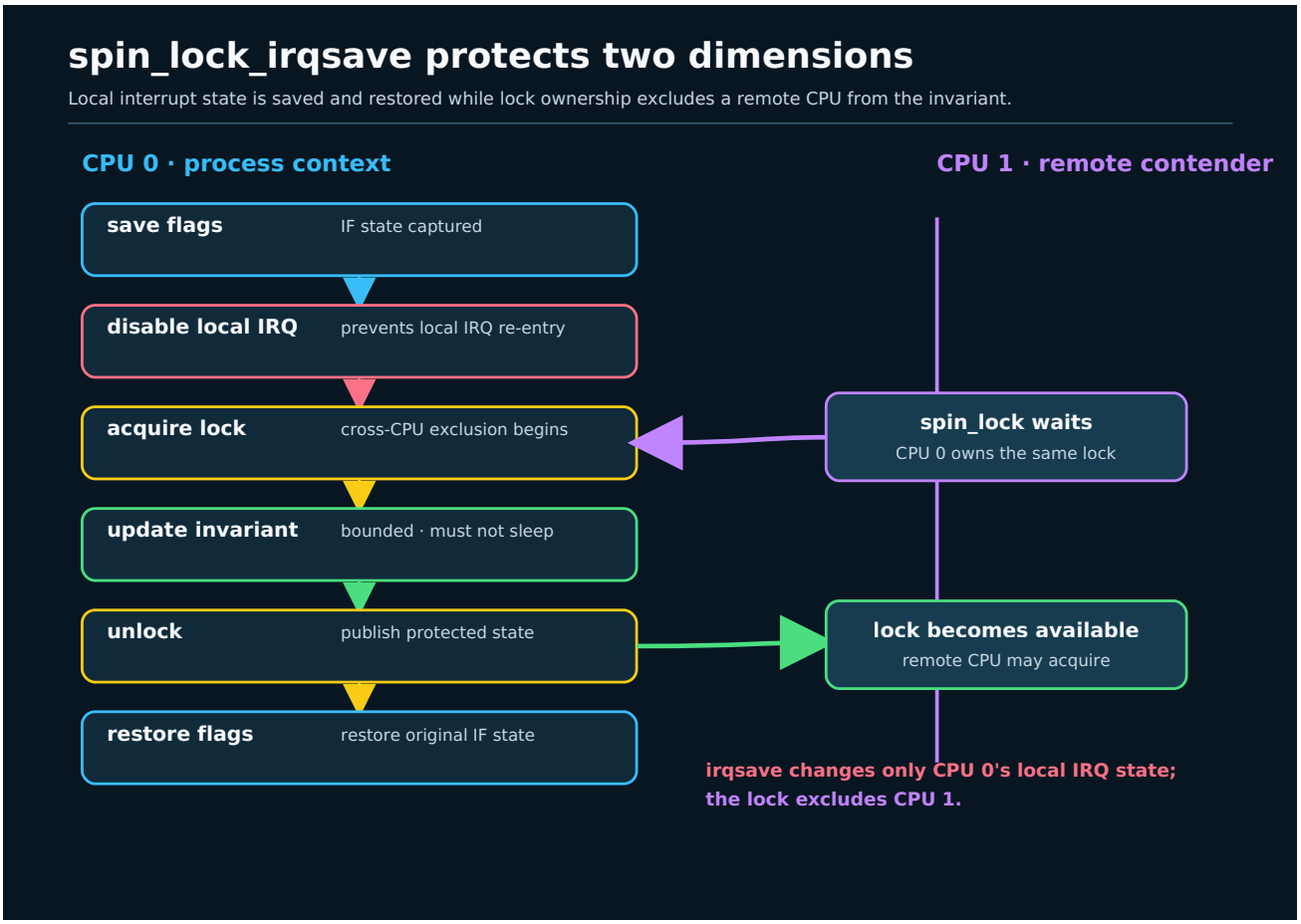
```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-002-atomics-memory-ordering-and-barriers.sh
```

- View lab source on GitHub

Day 3 — Spinlocks, IRQs, and Preemption

Target time: approximately 3 hours

> Driving question: why can a lock be correct against another CPU and still deadlock when an interrupt arrives on the CPU that owns it?



Why this mechanism exists

A shared invariant must never be observed half-updated. In process context, on another CPU, and in interrupt work, multiple execution contexts can reach the same object. A spinlock gives exactly one context permission to mutate the protected state. The owner is the CPU currently executing the successful acquire, not a task that may sleep and resume later: the owner must finish, publish the update, and unlock without blocking.

That statement has three independent requirements:

1. competitors on other CPUs must not enter the critical section;
2. this CPU must not be preempted into code that tries to take the same lock;
3. if a local hardirq or softirq can take it, the matching local source must be excluded before acquisition.

A plain spinlock handles the first and, on non-RT kernels, normally the second. It does not by itself prevent a hardirq from interrupting its owner. If that handler spins for the same lock, the only CPU able to release it is trapped in the interrupted critical section. `spin_lock_irqsave()` closes that local deadlock window while preserving the caller's previous interrupt state.

Syntax before mechanism

`spinlock_t *lock` is a pointer to a lock object. `unsigned long flags` receives the architecture interrupt-mask state. The macro writes that variable, so it is an output despite appearing as an ordinary argument. The matching `spin_unlock_irqrestore(lock, flags)` releases first and restores exactly the saved state; blindly enabling interrupts would break callers that entered with interrupts already disabled.

`spin_lock_bh()` disables local bottom-half processing around the lock. It is the process-context side of a lock also acquired by softirq code. It is not a hardirq mask. `spin_lock_irq()` disables local hardirqs without preserving a prior state and is appropriate only when the surrounding contract makes that state known. `raw_spinlock_t` and `spinlock_t` are deliberately distinct, especially under PREEMPT_RT.

`READ_ONCE()` and `WRITE_ONCE()` constrain compiler access width and coalescing; they are not mutual exclusion. Lock acquire and release supply ordering around data protected by the same lock. That does not make every unrelated lock operation a full global fence. When Linux needs ordering stronger than ordinary acquire semantics, it says so with helpers such as `smp_mb__after_spinlock()`.

Linux 0.12: interrupt exclusion before SMP locking

Read Chapter 8.6, `kernel/sched.c`, printed pages 351–372 of *A Heavily Commented Linux Kernel Source Code: Kernel Version 0.12*, and the Week 11 boundary in `references/EARLY_LINUX_012_MAP.md`. Also inspect `include/asm/system.h`, where `cli()` and `sti()` are direct x86 instructions. The early kernel is uniprocessor. Closing the interrupt flag prevents an interrupt handler from interleaving on that sole CPU, so many small critical regions need no SMP lock.

The durable lesson is local reentrancy: an interrupt can arrive between two C statements unless excluded. The historical boundary is equally important. Linux 0.12 has no second CPU concurrently touching the cache line, no modern spinlock API, no lockdep graph, no softirq/preemption accounting as Linux 6.12 uses it, and no PREEMPT_RT transformation. `cli` is not an SMP lock and `sti` is not a safe substitute for restoring saved flags.

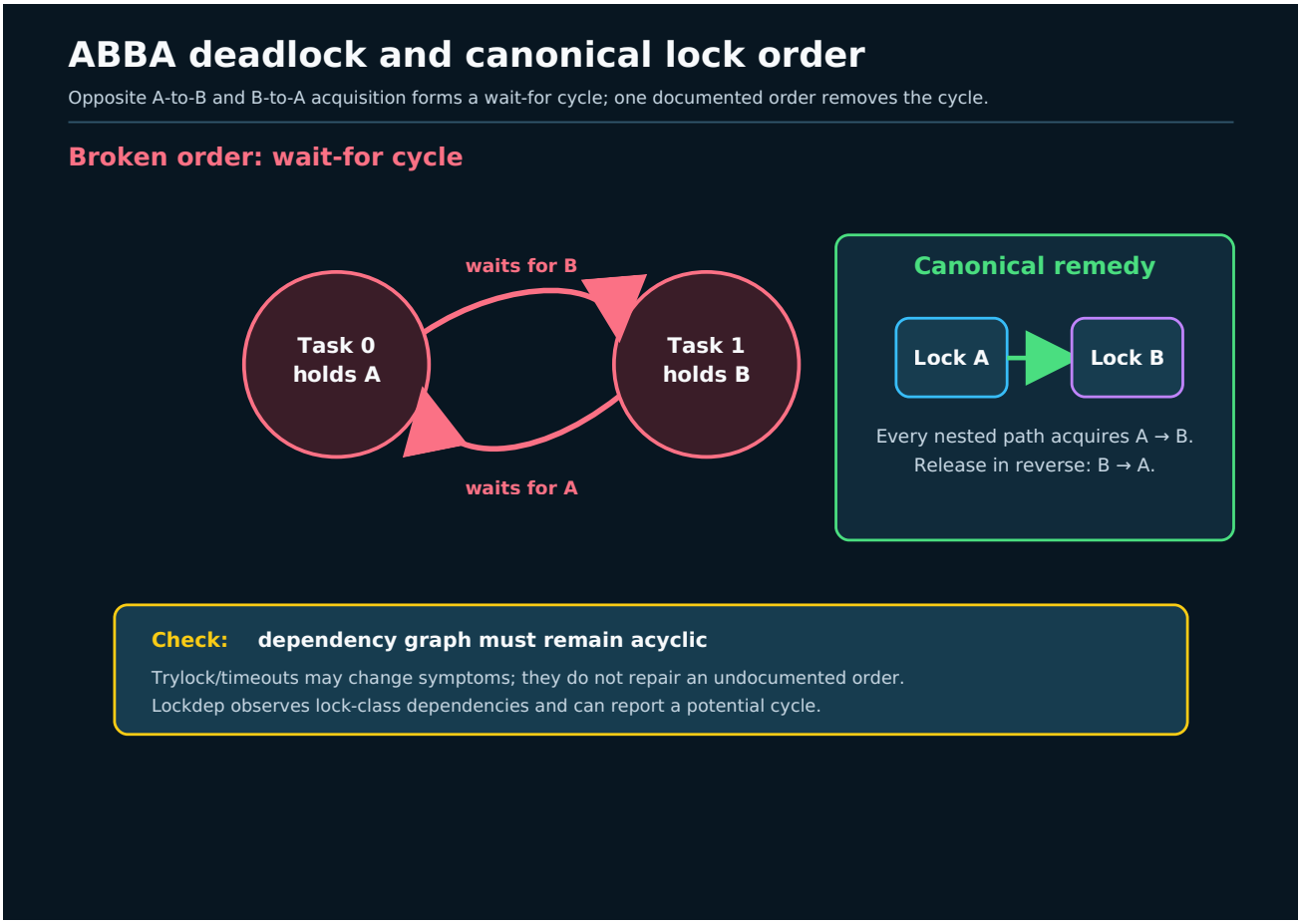
Evolution pressure

SMP made local interrupt masking insufficient: CPU 1 keeps running when CPU 0 clears its own IF bit. Kernel preemption added another local competitor. Deferred interrupt work required a distinction between hardirq and bottom-half exclusion. Large systems then made contention, cache-line bouncing, lock ordering, and diagnostics first-class concerns.

The resulting rule is contextual:

- use a plain spinlock only when no local interrupt context sharing the data can interrupt the owner;

- use the `_bh` form when process context shares with softirq/bottom-half work;
- use `_irqsave`/`_irqrestore` when hardirq sharing is possible or the incoming mask is not known;
- never sleep, fault deliberately, or call a blocking allocator while holding a spinning lock;
- use one global order for nested locks. If path A takes `L1 → L2` while path B takes `L2 → L1`, both CPUs can wait forever.



Lock ordering is part of the data structure's API, not a local style choice. Lockdep can discover observed order cycles, but a clean run does not prove that an unexecuted path is safe.

Actual ToyOS boundary

ToyOS implements interrupt entry and timer-driven switching, but no lock. `projects/toyos/src/arch.c:222-226` exposes `interrupts_enable()` as `sti`. `interrupt_handler()` at lines 240–258 increments `ticks`, sends the PIC EOI, and calls `scheduler_tick()`. `projects/toyos/src/tasks.c:130-160` changes the current task and can context-switch from that tick.

There is no `spinlock_t`, atomic lock word, IRQ-save helper, bottom-half facility, preemption counter, lock ordering, lockdep, or SMP CPU. The global `ticks` variable is `volatile`, which does not make it an SMP atomic or provide lock ordering. ToyOS therefore demonstrates that interrupt execution can interleave with task code; it does not demonstrate a correct Linux spinlock, an irqsave protocol, or a two-CPU deadlock.

Linux 6.12 pinned source

All excerpts are exact, contiguous source from commit `adc218676eef25575469234709c2d87185ca223a`. Line ranges refer to `/home/jd/Documents/linux-6.12`.

The ordinary interface reaches the raw implementation embedded in `spinlock_t`.

`include/linux/spinlock.h`, `spin_lock()` / `spin_lock_bh()`, lines 349–356

```
static __always_inline void spin_lock(spinlock_t *lock)
{
    raw_spin_lock(&lock->rlock);
}

static __always_inline void spin_lock_bh(spinlock_t *lock)
{
    raw_spin_lock_bh(&lock->rlock);
```

The irqsave macro type-checks writable flag storage and passes it down. The saved value belongs to this acquire and must reach the matching restore.

`include/linux/spinlock.h`, `raw_spin_lock_irqsave`, lines 239–245

```
#if defined(CONFIG_SMP) || defined(CONFIG_DEBUG_SPINLOCK)

#define raw_spin_lock_irqsave(lock, flags)          \
    do {                            \
        typecheck(unsigned long, flags);    \
        flags = _raw_spin_lock_irqsave(lock);   \
    } while (0)
```

The typed wrapper keeps the same pairing and rejects a non-spinlock argument through `spinlock_check`.

`include/linux/spinlock.h`, `spin_lock_irqsave`, lines 374–382

```
static __always_inline void spin_lock_irq(spinlock_t *lock)
{
    raw_spin_lock_irq(&lock->rlock);
}

#define spin_lock_irqsave(lock, flags)              \
do {                                    \
    raw_spin_lock_irqsave(spinlock_check(lock), flags); \
} while (0)
```

Locks normally order critical sections with acquire/release semantics. This commented litmus documents a site that explicitly strengthens ordering after acquisition; the forbidden result is narrower and more precise than saying “spinlocks are full barriers.”

`include/linux/spinlock.h`, memory-ordering example, lines 133–146

```
 *    { X = 0;  Y = 0; }
 *
 *    CPU0              CPU1
 *
 *    WRITE_ONCE(X, 1);     WRITE_ONCE(Y, 1);
 *    spin_lock(S);         smp_mb();
 *    smp_mb__after_spinlock(); r1 = READ_ONCE(X);
 *    r0 = READ_ONCE(Y);
 *    spin_unlock(S);
 *
 *      it is forbidden that CPU0 does not observe CPU1's store to Y (r0 = 0)
 *      and CPU1 does not observe CPU0's store to X (r1 = 0); see the comments
 *      preceding the call to smp_mb__after_spinlock() in __schedule() and in
 *      try_to_wake_up().
```

The raw acquire reaches the architecture lock and annotates ownership for debugging. This is spinning state, not a scheduler wait queue.

`include/linux/spinlock.h`, `do_raw_spin_lock()`, lines 184–189

```
static inline void do_raw_spin_lock(raw_spinlock_t *lock) __acquires(lock)
{
    __acquire(lock);
    arch_spin_lock(&lock->raw_lock);
    mmiowb_spin_lock();
}
```

PREEMPT_RT changes regular `spinlock_t`: this header routes it through an rt-mutex-based implementation. The shown irqsave flags become zero rather than an ordinary hardirq save. Therefore claims about “all spinlocks disable preemption and spin with IRQs off” are false across configurations.

`include/linux/spinlock_rt.h`, RT lock variants, lines 85–102

```
static __always_inline void spin_lock_bh(spinlock_t *lock)
{
    /* Investigate: Drop bh when blocking ? */
    local_bh_disable();
    rt_spin_lock(lock);
}

static __always_inline void spin_lock_irq(spinlock_t *lock)
{
    rt_spin_lock(lock);
}

#define spin_lock_irqsave(lock, flags)           \
    do {                             \
        typecheck(unsigned long, flags);     \
        flags = 0;               \
        spin_lock(lock);             \
    } while (0)
```

`raw_spinlock_t` remains the primitive for truly atomic low-level sections on PREEMPT_RT; regular `spinlock_t` is transformed to improve latency. This lesson does not grant permission to sleep while holding either API, nor does it claim that an arbitrary driver becomes hard real-time merely by running an RT-patched kernel.

Reason through the invariant

Suppose task context takes `device->lock` and is interrupted locally.

- With plain `spin_lock`, a hardirq handler taking the same lock spins forever: the interrupted task is the owner and cannot run to unlock.
- With `spin_lock_irqsave`, the hardirq cannot enter on this CPU between mask save and restore. A handler on another CPU may spin, but the owner CPU can continue and release.
- With only `spin_lock_bh`, softirq execution is deferred locally, but a hardirq can still arrive.
- With two locks, irqsave does not cure ABBA. Every path still needs the same nested order.

The protected writes happen before unlock; a later successful acquire of the same lock observes the critical section through the lock's ordering contract. Data accessed outside that protocol needs its own atomic, RCU, sequence-count, or other documented ordering. A spinlock does not retroactively protect an unlocked read.

Bounded lab: analogy, not kernel-path proof

Use the Day 3 exercise under `projects/linux-kernel-sync`. Its userspace analogs/simulators may model a test-and-set owner, saved “IRQ enabled” state, an injected local handler, and an ABBA timeout. Bound every run with a timeout and report deadlock as a modeled state, not by hanging the course runner.

The lab can show:

- one modeled owner enters at a time;
- restoring the saved state differs from unconditional enable;
- an injected same-CPU handler deadlocks in the unsafe model;
- a fixed lock order completes in the bounded model.

It cannot prove that the host entered `spin_lock_irqsave`, disabled a real interrupt line, exercised Linux lockdep, ran PREEMPT_RT, or obtained any specific architecture barrier. Those claims come only from the pinned source and an identified kernel build. Record `uname -r`; do not infer that the host runs this 6.12 commit.

Connections and limits

Week 11 supplied races, acquire/release reasoning, and the warning that `volatile` is not synchronization. Week 14 supplied hardirq, softirq, and preemption context: those contexts determine which lock suffix is required. Week 15 supplies scheduling, but a non-RT spinlock section must finish without sleeping into that scheduler; PREEMPT_RT's transformed regular locks are a configuration-specific exception in implementation, not a blanket API change.

Do not use a spinlock for long work, I/O waits, userspace copies that may fault, or a condition that requires another task to run. Do not use irqsave merely as an SMP substitute, and do not use a mutex in hardirq or softirq context. The excerpts establish Linux 6.12 at the named commit only. Later upstream is **potentially changed** until checked at a named revision.

Completion check

Explain one shared object as:

```
invariant → possible CPU/IRQ/softirq competitors → chosen lock variant
→ acquire ordering → bounded non-sleeping mutation → release ordering
→ exact interrupt-state restoration
```

Then draw an ABBA cycle and remove it with one global order. Label every claim `Linux 0.12 historical`, `ToyOS actual`, `Linux 6.12 pinned`, `userspace model`, or `not established`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Spinlocks Irqs And Preemption

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

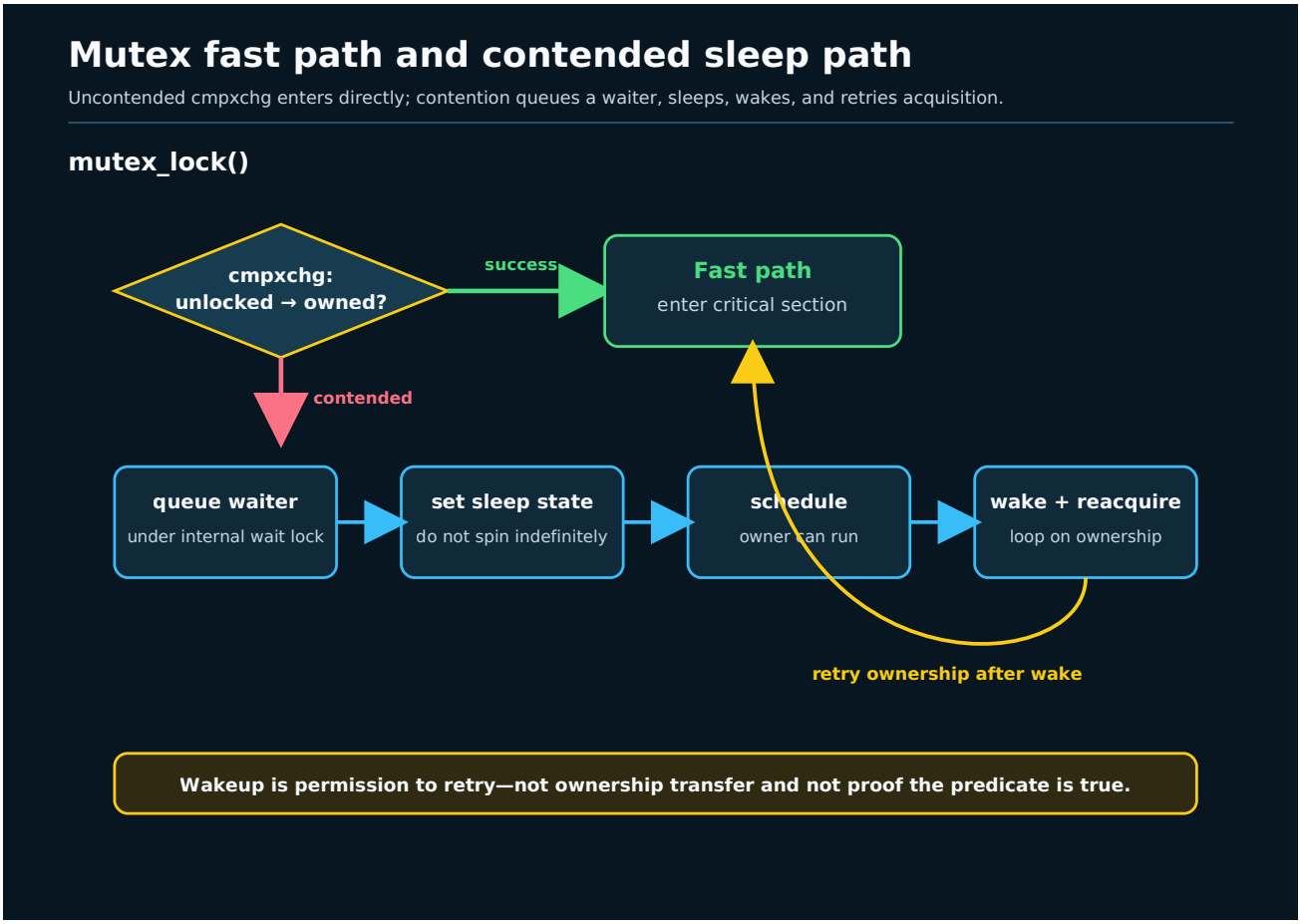
```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-003-spinlocks-irqs-and-preemption.sh
```

- View lab source on GitHub

Day 4 — Mutexes, rtmutexes, and Sleeping Locks

Target time: approximately 3 hours

> Driving question: when waiting for a lock may take an arbitrary time, why should the contender stop running, and how can priority inheritance keep the owner from being stranded behind unrelated work?



Why sleeping locks exist

Spinning is useful only when the owner can run and release soon. If a critical section may wait for memory, perform a blocking operation, or simply suffer long contention, a spinning CPU burns execution time while making no progress. A mutex moves a losing task onto a waiter list and lets the scheduler run something else. The cost is larger machinery: task state, queueing, wakeup, ownership, cancellation, and a retry protocol.

The invariant is exclusive ownership by one task. Unlike the CPU-oriented spinlock rule, mutex ownership follows the task: only the owner unlocks, the task may sleep while waiting to acquire, and acquisition is legal only in sleepable process context. “A mutex can sleep” means the **acquire operation may block**. It is not permission to call arbitrary sleeping functions while a particular subsystem's mutex is held; higher-level lifetime and lock-order rules still apply.

Syntax before mechanism

`atomic_long_try_cmpxchg_acquire(&lock->owner, &zero, curr)` compares the owner word with zero. On success it installs `current`, returns true, and supplies acquire ordering. On failure it updates the expected value and sends the caller toward the slow path.

`__sched` marks a function that may schedule. `might_sleep()` is a debugging and voluntary-preemption annotation, not the mechanism that queues a waiter. `set_current_state(state)` publishes a sleep state with ordering. A `for (;;)` loop is required because wakeup does not grant ownership: the task must retry. `wake_q_add()` builds a wake batch under the internal wait lock; `wake_up_q()` runs after that lock is dropped.

An rtmutex is a mutex with priority-inheritance bookkeeping. “PI” means a higher-priority waiter can donate effective priority to an owner that blocks it. It does not mean the lock is automatically available, that deadlines are guaranteed, or that a userspace process has permission to use a privileged real-time scheduling class.

Linux 0.12: whole sleep/wakeup shape, not a modern mutex

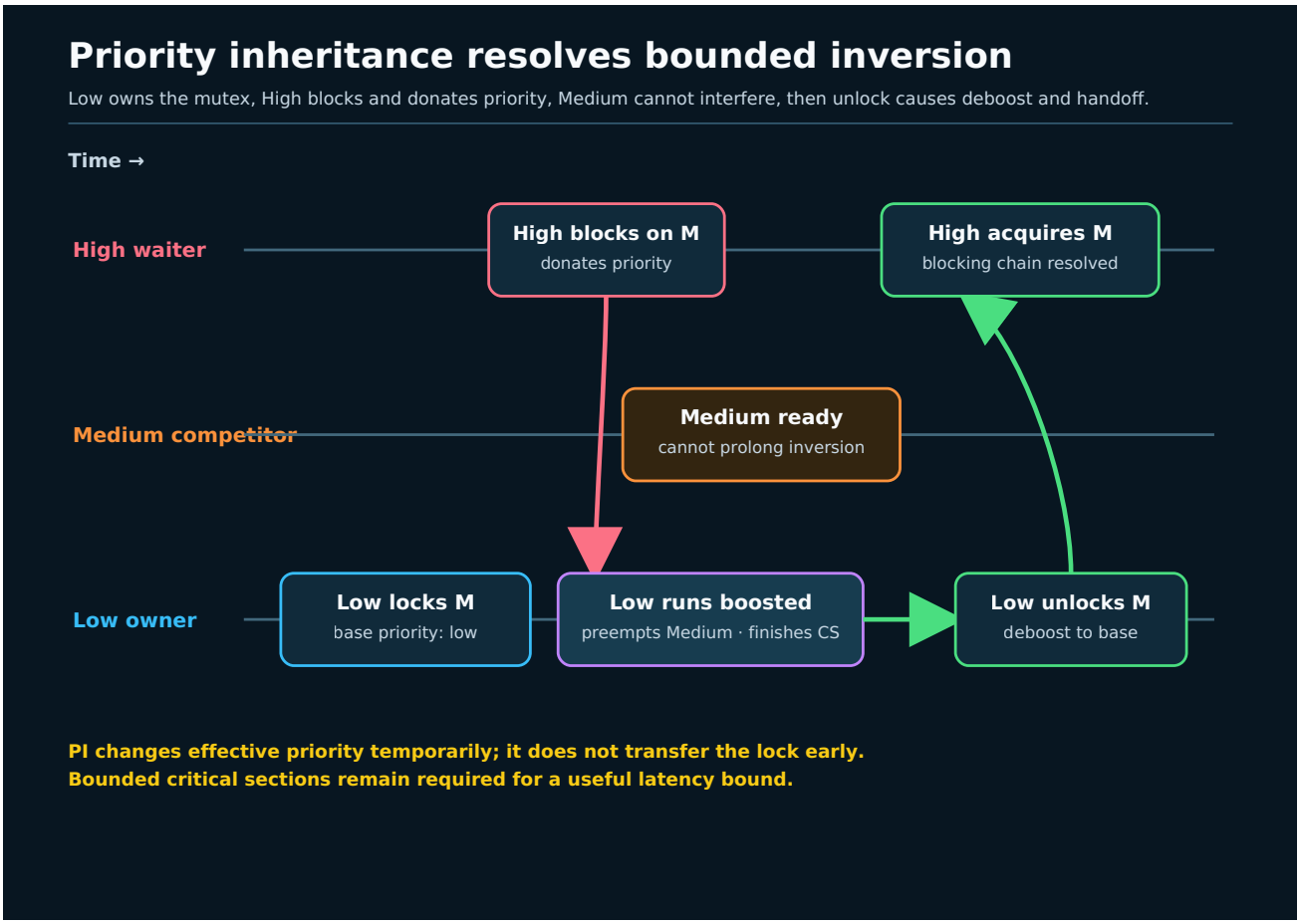
Read Chapter 8.6, `kernel/sched.c`, printed pages 351–372 in the local commented Linux 0.12 PDF, as directed by the Week 11 section of `references/EARLY_LINUX_012_MAP.md`. The `sleep_on`, `interruptible_sleep_on`, and `wake_up` helpers expose a compact transition: set task state, call `schedule()`, and later mark a waiter `TASK_RUNNING`. The pipe example mapped at Chapter 12.11, printed pages 802–806, supplies a resource condition that makes a task wait.

The stable mechanism is relinquish-and-retry: a task unable to proceed stops consuming the CPU, another context changes the condition, and wakeup makes the waiter runnable. The implementation is historical. A single wait pointer and uniprocessor interrupt exclusion do not provide Linux 6.12 mutex ownership, FIFO waiter lists, handoff flags, SMP ordering, lockdep, optimistic spinning, or priority inheritance. Do not rename `sleep_on()` a mutex.

Evolution pressure

SMP made owner and waiter transitions concurrent. Many waiters required a real list and cancellation rules. Fast uncontended locks needed to stay close to one atomic operation, while contention required a separate slow path. Unlock had to prevent lost wakeups and unfair lock stealing without waking under an internal raw spinlock.

Priority inversion added a different pressure. If high-priority task H waits on a lock held by low-priority L, and medium-priority M keeps preempting L, then H is indirectly delayed by M. PI temporarily raises L's effective priority to that of its most important donor, and can propagate through a chain if L itself blocks on another rtmutex. Unlock removes or replaces the donation and recalculates priority—deboosting is as important as boosting.



Week 15 explains how that effective priority interacts with scheduler classes, runqueues, preemption, and wakeup. This lesson stops at the locking side of that boundary. It makes no claim that the learner can set `SCHED_FIFO`, that the host grants `CAP_SYS_NICE`, or that a timing sample proves deterministic real-time behavior.

Actual ToyOS boundary

`projects/toyos/src/tasks.c:4-8` defines runnable, running, and blocked states. At lines 53–70, `task_finish()` marks the current task blocked only after its work is complete and calls the private scheduler. At lines 130–153, `schedule()` skips a blocked candidate and switches between two ring-0 tasks.

There is no mutex object, owner word, waiter list, contention loop, wake operation, semaphore, rwsem, completion, priority field, PI tree, or deboost. ToyOS can show that a blocked state removes a task from simple selection and that a saved continuation may later resume. It cannot show a task sleeping *for a lock*, a lost-wakeup-safe mutex protocol, or a scheduler response to priority donation.

Linux 6.12 pinned: the uncontended path

All excerpts are exact, contiguous source from commit `adc218676eef25575469234709c2d87185ca223a`. Line ranges identify `/home/jd/Documents/linux-6.12`.

The fast path installs `current` into an empty owner word with acquire ordering. No wait-list lock or scheduler call is needed.

`kernel/locking/mutex.c`, `__mutex_trylock_fast()`, lines 162–174

```
/*
 * Optimistic trylock that only works in the uncontended case. Make sure to
 * follow with a __mutex_trylock() before failing.
 */
static __always_inline bool __mutex_trylock_fast(struct mutex *lock)
{
    unsigned long curr = (unsigned long)current;
    unsigned long zero = 0UL;

    if (atomic_long_try_cmpxchg_acquire(&lock->owner, &zero, curr))
        return true;

    return false;
```

The public operation states sleepability before branching. Failure of the optimistic attempt is not final failure; it invokes the full protocol.

`kernel/locking/mutex.c`, `mutex_lock()`, lines 275–288

```
 * (The CONFIG_DEBUG_MUTEXES .config option turns on debugging
 * checks that will enforce the restrictions and will also do
 * deadlock debugging)
 *
 * This function is similar to (but not equivalent to) down().
 */
void __sched mutex_lock(struct mutex *lock)
{
    might_sleep();

    if (!__mutex_trylock_fast(lock))
        __mutex_lock_slowpath(lock);
}
EXPORT_SYMBOL(mutex_lock);
```

Linux 6.12 pinned: queue, sleep, retry, wake

The ordinary slow path appends in FIFO order, then publishes task state. The internal `wait_lock` serializes list and handoff state; it is not the mutex being requested.

`kernel/locking/mutex.c`, `__mutex_lock_common()`, lines 638–654

```
    lock_contended(&lock->dep_map, ip);

    if (!use_ww_ctx) {
        /* add waiting tasks to the end of the waitqueue (FIFO): */
        __mutex_add_waiter(lock, &waiter, &lock->wait_list);
    } else {
        /*
         * Add in stamp order, waking up waiters that must kill
         * themselves.
         */
        ret = __ww_mutex_add_waiter(&waiter, lock, ww_ctx);
        if (ret)
            goto err_early_kill;
    }

    set_current_state(state);
    trace_contention_begin(lock, LCB_F_MUTEX);
```

The contender drops the internal raw lock before scheduling. After wakeup it re-establishes state and retries acquisition or handoff. This loop is why “wakeup transfers the mutex” is an unsafe simplification.

`kernel/locking/mutex.c`, `__mutex_lock_common()`, lines 683–695

```
        raw_spin_unlock(&lock->wait_lock);
        schedule_preempt_disabled();

        first = __mutex_waiter_is_first(lock, &waiter);

        set_current_state(state);
        /*
         * Here we order against unlock; we must either see it change
         * state back to RUNNING and fall through the next schedule(),
         * or we must see its unlock and acquire.
         */
        if (__mutex_trylock_or_handoff(lock, first))
            break;
```

Unlock selects the first waiter while protected, optionally records a direct handoff, drops `wait_lock`, and only then performs the wake batch.

`kernel/locking/mutex.c`, `__mutex_unlock_slowpath()`, lines 937–956

```
    raw_spin_lock(&lock->wait_lock);
    debug_mutex_unlock(lock);
    if (!list_empty(&lock->wait_list)) {
        /* get the first entry from the wait-list: */
        struct mutex_waiter *waiter =
            list_first_entry(&lock->wait_list,
                     struct mutex_waiter, list);

        next = waiter->task;

        debug_mutex_wake_waiter(lock, waiter);
        wake_q_add(&wake_q, next);
    }

    if (owner & MUTEX_FLAG_HANDOFF)
        __mutex_handoff(lock, next);

    raw_spin_unlock(&lock->wait_lock);

    wake_up_q(&wake_q);
```

Linux 6.12 pinned: PI and deboost

When an rtmutex waiter becomes the top waiter, Linux replaces the owner's PI donor, recalculates effective priority, and notes whether the owner is itself blocked so a chain walk may be needed.

`kernel/locking/rtmutex.c`, `task_blocks_on_rt_mutex()`, lines 1261–1276

```
    raw_spin_lock(&owner->pi_lock);
    if (waiter == rt_mutex_top_waiter(lock)) {
        rt_mutex_dequeue_pi(owner, top_waiter);
        rt_mutex_enqueue_pi(owner, waiter);

        rt_mutex_adjust_prio(lock, owner);
        if (owner->pi_blocked_on)
            chain_walk = 1;
    } else if (rt_mutex_cond_detect_deadlock(waiter, chwalk)) {
        chain_walk = 1;
    }

    /* Store the lock on which owner is blocked or NULL */
    next_lock = task_blocked_on_lock(owner);

    raw_spin_unlock(&owner->pi_lock);
```

On unlock, the departing owner loses this donor and is deboosted before the top waiter is woken. Keeping new arrivals on the slow path prevents a lower-priority task from stealing ahead of that waiter.

`kernel/locking/rtmutex.c`, wakeup/deboost path, lines 1319–1337

```
    /*
     * Remove it from current->pi_waiters and deboost.
     *
     * We must in fact deboost here in order to ensure we call
     * rt_mutex_setprio() to update p->pi_top_task before the
     * task unblocks.
     */
    rt_mutex_dequeue_pi(current, waiter);
    rt_mutex_adjust_prio(lock, current);

    /*
     * As we are waking up the top waiter, and the waiter stays
     * queued on the lock until it gets the lock, this lock
     * obviously has waiters. Just set the bit here and this has
     * the added benefit of forcing all new tasks into the
     * slow path making sure no task of lower priority than
     * the top waiter can steal this lock.
     */
    lock->owner = (void *) RT_MUTEX_HAS_WAITERS;
```

PI bounds one cause of inversion: an owner preempted by work below the donor's effective priority. It cannot bound device latency, non-preemptible regions, interrupt storms, a lock chain outside the PI protocol, or arbitrary critical section length. It is a mechanism, not a privileged scheduling guarantee.

Boundaries among sleeping primitives

A mutex represents exclusive ownership and must be unlocked by its owner. A counting semaphore represents available units; `down()` / `up()` are not a drop-in ownership discipline. An rwsem permits multiple readers or one writer and is useful only when its read concurrency offsets extra fairness and cache costs. A completion represents an event or handoff—“this operation finished”— not protection of a mutable invariant. Its waiter normally calls `wait_for_completion*()` and the producer calls `complete()` or `complete_all()`.

Choose by the fact being represented:

- **one owner protects state:** mutex;
- **N fungible units:** semaphore;
- **many read-side holders, one writer:** rwsem;
- **one operation reaches a milestone:** completion;
- **atomic-context, very short invariant:** spinlock from Day 3.

None of the sleeping primitives is legal from hardirq or softirq context. Completions can be signaled from atomic contexts where their implementation permits it, but waiting is the sleeping side and requires process context.

Bounded lab: analogy, not kernel-path proof

Use the Day 4 programs under `projects/linux-kernel-sync`. A bounded userspace mutex analog may force one thread to hold a `pthread_mutex_t`, observe that a contender stops making progress until unlock, and record wake-to-acquire time. A PI simulator may model H waiting on L while M is runnable, then compare a schedule with and without donation and deboost. Put a timeout around every run.

These exercises can establish their own userspace ordering and simulator state. They cannot prove that the kernel used `kernel/locking/mutex.c`, that a futex took a particular slow path, that the scheduler performed Linux rtmutex PI, or that the process received real-time policy. Do not request elevated priority, capabilities, or host-wide scheduler changes. Keep `kernel_path_claim=none` in the result.

Connections, misuse, and limits

Week 11's condition-variable loop explains why waiters recheck instead of assuming wakeup grants ownership. Week 14 determines sleep legality: process context may block; hardirq, softirq, and NMI may not. Week 15 owns the next decision after a waiter becomes runnable and explains how effective priority changes selection.

A mutex is inappropriate for interrupt context, recursively locking the same non-recursive mutex, or data read without participating in the lock protocol. An rtmutex is inappropriate as a claim of hard real-time completion. A semaphore is inappropriate when owner-only unlock and lockdep ownership are the invariant. A completion is inappropriate as a reusable data lock without a separate generation/lifetime design.

The excerpts prove only the named Linux 6.12 commit. A host run must record `uname -r`; later upstream and downstream RT trees are **potentially changed** until checked at named revisions.

Completion check

Narrate both paths:

```
unowned → acquire cmpxchg → protected work → release

owned → enqueue + publish sleep state → schedule → unlock selects waiter
→ wake → retry/handoff → acquire
```

Then add `H waits on L → donate → L runs → unlock → deboost → H retries` and mark where Week 15 scheduler policy begins. Label each assertion `Linux 0.12 historical`, `ToyOS actual`, `Linux 6.12 pinned`, `userspace analog`, or `not established`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mutexes Rtmutexes And Sleeping Locks

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

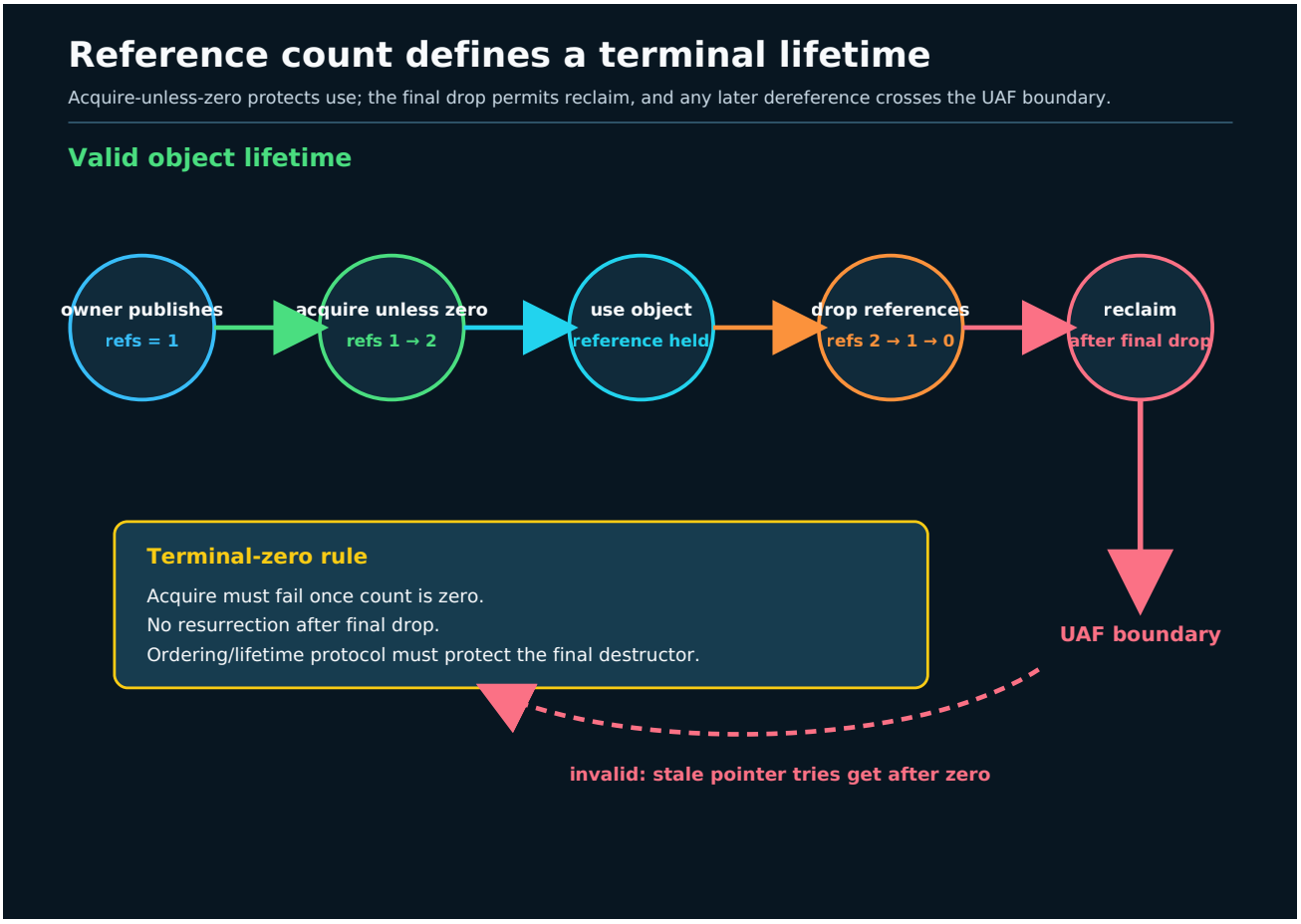
```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-004-mutexes-rtmutexes-and-sleeping-locks.sh
```

- View lab source on GitHub

Day 5 — Refcounts, Seqlocks, and Per-CPU State

Target time: approximately 3 hours

> Driving question: is the bug that two contexts update an object together, that one context frees it while another still uses it, or that a global counter forces every CPU onto the same cache line?



Why these are three different mechanisms

Exclusion, lifetime, consistent snapshots, and locality solve different problems.

- A lock prevents conflicting operations from overlapping.
- A reference count prevents destruction while a valid reference exists.
- A sequence counter lets readers detect that a snapshot overlapped a writer and retry.
- Per-CPU storage gives each CPU a local instance to reduce sharing.

None substitutes automatically for another. A refcount of two does not stop both holders mutating the object. A mutex around lookup does not keep the object alive after unlock unless the lookup acquires a reference. A seqcount does not serialize two writers. A per-CPU counter is not a coherent global total until an aggregation protocol defines what “now” means.

The first debugging question is therefore **which invariant failed?** Only then choose a primitive.

Syntax before mechanism

`refcount_t` wraps an atomic counter with misuse checks and saturation semantics. `refcount_inc()` means the caller already owns a valid reference. Acquiring from a lookup where zero is possible needs a documented `*_inc_not_zero()`/RCU/lock protocol. `refcount_dec_and_test()` returns true only to the last releaser, which then destroys the object.

`container_of(kref, struct object, ref)` recovers the containing object in a release callback. A `kref` is commonly embedded in that object; `kref_get()` and `kref_put()` encode the increment/final-release pattern.

A seqcount reader saves an even sequence, copies all fields, and asks whether the sequence changed. An odd value means a writer is active. A retry discards the entire snapshot. The writer must already be serialized and must not be preempted into indefinite suspension while readers spin.

`DEFINE_PER_CPU(type, name)` declares one instance per possible CPU. `this_cpu_*` accesses the current CPU's instance with the API's stated preemption/interrupt guarantees. A pointer obtained by `this_cpu_ptr()` can become the wrong CPU's pointer after migration unless migration or preemption is held off for its use.

Linux 0.12: counts and global state before modern hardening

Use the precise readings in `references/EARLY_LINUX_012_MAP.md`: Chapter 8.6, `kernel/sched.c`, printed pages 351–372, shows one global task view; Chapter 12.14, `fs/open.c`, printed pages 817–825, shows `f_count` ownership in the fixed global file table; Chapter 12.11, `fs/pipe.c`, printed pages 802–806, shows inode counts participating in pipe endpoint lifetime.

The durable idea is that an object remains allocated while owners still refer to it, and the final release changes what cleanup is legal. The historical implementation is a plain integer in a uniprocessor kernel, often protected by interrupt exclusion or surrounding global structure. Linux 0.12 cannot teach `refcount_t` saturation, modern atomic ordering, kref lookup rules, seqcounts, per-CPU allocation, migration safety, or SMP cache-line locality.

The map explicitly warns that 0.12 has no SMP memory-ordering problem, RCU, or per-CPU runqueues. Preserve that boundary: an early `f_count++` is a useful lifetime picture, not current lock-free reference-count guidance.

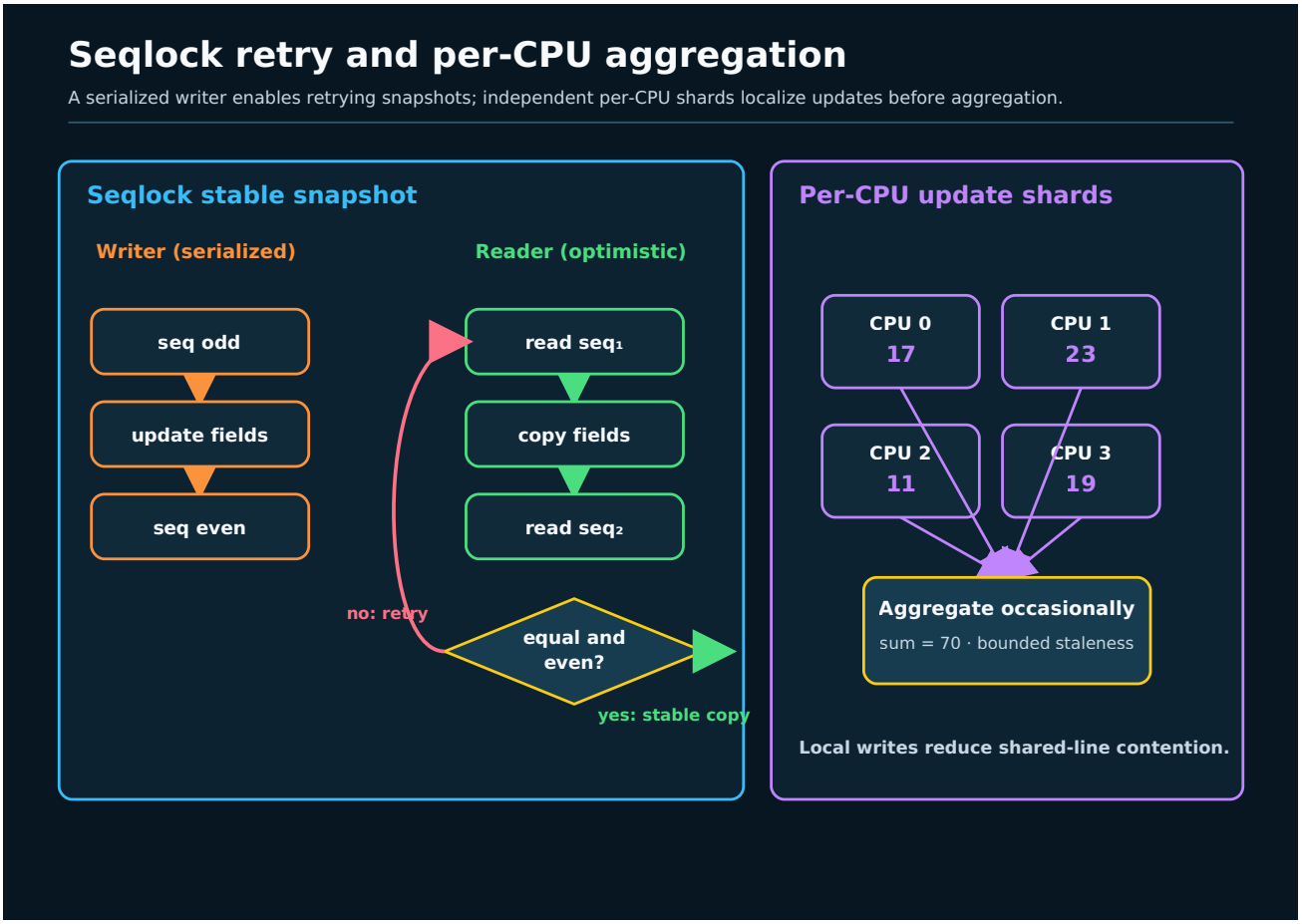
Evolution pressure

SMP exposed three scalability and safety failures.

First, wrapping a small or unchecked atomic reference count through zero can turn a misuse into premature free and use-after-free. `refcount_t` adds warnings, forbidden transitions, and saturation so overflow tends toward a leak rather than resurrection or premature destruction.

Second, read-mostly state sometimes needs a multi-field snapshot but cannot afford a lock in every reader. Sequence counters make readers optimistic: copy, validate, retry. Writers still serialize and publish an odd/even generation around the update.

Third, a single hot global counter makes every CPU acquire exclusive ownership of one cache line. Per-CPU state moves the common update to local storage and pays aggregation cost later. This improves locality; it does not remove the need to pin execution while deriving and using a current-CPU address.



Actual ToyOS boundary

ToyOS has neither of the modern mechanisms. In `projects/toyos/src/process.c:65-70`, `init_process`, `pipe_data`, `pipe_length`, and `pipe_read_position` are single global objects. The pipe read/write helpers at lines 150–167 update those globals without a lock, refcount, seqcount, or per-CPU shard. Only one user process exists.

`projects/toyos/src/tasks.c:20-22` has a global two-element task array, one `current` pointer, and completion flags. `task_finish()` at lines 53–70 marks a task done; it does not decrement an object reference or invoke a destructor. `volatile` on the flags and timer tick constrains compiler treatment but does not supply an SMP lifetime protocol.

ToyOS therefore exposes the pressure point—a global state design that would need synchronization before SMP—but implements no `refcount_t`, `kref`, seqcount retry loop, CPU-local allocation, migration control, or aggregation. Do not infer those mechanisms from one-CPU success.

Linux 6.12 pinned: hardened lifetime

All excerpts are exact, contiguous source from commit `adc218676eef25575469234709c2d87185ca223a`. Line ranges identify `/home/jd/Documents/linux-6.12`.

The design deliberately saturates instead of wrapping. Saturation can leak an object after misuse; that is safer than allowing a wrapped count to authorize an erroneous final free.

`include/linux/refcount.h`, saturation comment, lines 8–24

```
 * Saturation semantics
 * ====================
 *
 * refcount_t differs from atomic_t in that the counter saturates at
 * REFCOUNT_SATURATED and will not move once there. This avoids wrapping the
 * counter and causing 'spurious' use-after-free issues. In order to avoid the
 * cost associated with introducing cmpxchg() loops into all of the saturating
 * operations, we temporarily allow the counter to take on an unchecked value
 * and then explicitly set it to REFCOUNT_SATURATED on detecting that underflow
 * or overflow has occurred. Although this is racy when multiple threads
 * access the refcount concurrently, by placing REFCOUNT_SATURATED roughly
 * equidistant from 0 and INT_MAX we minimise the scope for error:
 *
 *                            INT_MAX     REFCOUNT_SATURATED   UINT_MAX
 *   0                         (0x7fff_ffff)    (0xc000_0000)    (0xffff_ffff)
 *   +--------------------------------+----------------+----------------+
 *                                     <---------- bad value! ---------->
```

An ordinary increment assumes the object is already stable because the caller owns a reference. It intentionally provides no general publication ordering.

`include/linux/refcount.h`, `refcount_inc()`, lines 244–259

```
/**
 * refcount_inc - increment a refcount
 * @r: the refcount to increment
 *
 * Similar to atomic_inc(), but will saturate at REFCOUNT_SATURATED and WARN.
 *
 * Provides no memory ordering, it is assumed the caller already has a
 * reference on the object.
 *
 * Will WARN if the refcount is 0, as this represents a possible use-after-free
 * condition.
 */
static inline void refcount_inc(refcount_t *r)
{
    __refcount_inc(r, NULL);
}
```

The final decrement supplies release ordering for prior accesses and acquire ordering on the zero result so destruction follows them.

`include/linux/refcount.h`, `refcount_dec_and_test()`, lines 310–325

```
/**
 * refcount_dec_and_test - decrement a refcount and test if it is 0
 * @r: the refcount
 *
 * Similar to atomic_dec_and_test(), it will WARN on underflow and fail to
 * decrement when saturated at REFCOUNT_SATURATED.
 *
 * Provides release memory ordering, such that prior loads and stores are done
 * before, and provides an acquire ordering on success such that free()
 * must come after.
 *
 * Return: true if the resulting refcount is 0, false otherwise
 */
static inline __must_check bool refcount_dec_and_test(refcount_t *r)
{
    return __refcount_dec_and_test(r, NULL);
```

`kref_put()` packages “decrement, then one final release callback.” The callback is object-specific because destruction may remove registrations, release children, and only then free storage.

`include/linux/kref.h`, `kref_put()`, lines 49–69

```
 * kref_put - decrement refcount for object.
 * @kref: object.
 * @release: pointer to the function that will clean up the object when the
 *       last reference to the object is released.
 *       This pointer is required, and it is not acceptable to pass kfree
 *       in as this function.
 *
 * Decrement the refcount, and if 0, call release().
 * Return 1 if the object was removed, otherwise return 0.  Beware, if this
 * function returns 0, you still can not count on the kref from remaining in
 * memory.  Only use the return value if you want to see if the kref is now
 * gone, not present.
 */
static inline int kref_put(struct kref *kref, void (*release)(struct kref *kref))
{
    if (refcount_dec_and_test(&kref->refcount)) {
        release(kref);
        return 1;
    }
    return 0;
}
```

The normal embedded pattern is:

```
struct object {
    struct kref ref;
    /* object fields */
};

static void object_release(struct kref *ref)
{
    struct object *obj = container_of(ref, struct object, ref);
    /* unregister/release owned resources, then free obj */
}
```

This proposed pattern illustrates containment; it is not an additional pinned excerpt. A lookup must keep the object stable while acquiring its reference. Incrementing memory after another CPU has freed it is not repaired by an atomic operation.

Linux 6.12 pinned: consistent retry

The reader-side validation applies a read barrier, compares with the starting sequence, and requires the caller to discard an invalid snapshot.

`include/linux/seqlock.h`, sequence retry, lines 365–389

```
static inline int do___read_seqcount_retry(const seqcount_t *s, unsigned start)
{
    kcsan_atomic_next(0);
    return unlikely(READ_ONCE(s->sequence) != start);
}

/**
 * read_seqcount_retry() - end a seqcount_t read critical section
 * @s: Pointer to seqcount_t or any of the seqcount_LOCKNAME_t variants
 * @start: count, from read_seqcount_begin()
 *
 * read_seqcount_retry closes the read critical section of given
 * seqcount_t.  If the critical section was invalid, it must be ignored
 * (and typically retried).
 *
 * Return: true if a read section retry is required, else false
 */
#define read_seqcount_retry(s, start)                   \
    do_read_seqcount_retry(seqprop_const_ptr(s), start)

static inline int do_read_seqcount_retry(const seqcount_t *s, unsigned start)
{
    smp_rmb();
    return do___read_seqcount_retry(s, start);
}
```

The write-side contract is explicit: writers are serialized and non-preemptible, with IRQ/BH exclusion added when readers can interrupt them. `seqcount_t` does not create writer serialization by itself; associated-lock variants can assert and help provide that contract, while `seqlock_t` embeds a spinlock.

`include/linux/seqlock.h`, `write_seqcount_begin()`, lines 459–480

```
 * write_seqcount_begin() - start a seqcount_t write side critical section
 * @s: Pointer to seqcount_t or any of the seqcount_LOCKNAME_t variants
 *
 * Context: sequence counter write side sections must be serialized and
 * non-preemptible. Preemption will be automatically disabled if and
 * only if the seqcount write serialization lock is associated, and
 * preemptible.  If readers can be invoked from hardirq or softirq
 * context, interrupts or bottom halves must be respectively disabled.
 */
#define write_seqcount_begin(s)                     \
do {                                    \
    seqprop_assert(s);                      \
                                \
    if (seqprop_preemptible(s))                 \
        preempt_disable();                  \
                                \
    do_write_seqcount_begin(seqprop_ptr(s));            \
} while (0)

static inline void do_write_seqcount_begin(seqcount_t *s)
{
    do_write_seqcount_begin_nested(s, 0);
```

A seqcount is inappropriate when readers cannot cheaply retry, when the snapshot contains pointers whose targets may be freed during the read, when a writer can sleep in the odd-sequence interval, or when write contention is high. Lifetime still needs RCU, references, or a lock-based protocol.

Linux 6.12 pinned: CPU locality and migration

These helpers pin preemption while returning and using a current-CPU pointer. The paired put re-enables it. The pointer must not escape past that boundary without another rule keeping the selected CPU stable.

`include/linux/percpu-defs.h`, `get_cpu_var()` / `get_cpu_ptr()`, lines 275–295

```
#define get_cpu_var(var)                        \
(*({                                    \
    preempt_disable();                      \
    this_cpu_ptr(&var);                     \
}))

/*
 * The weird & is necessary because sparse considers (void)(var) to be
 * a direct dereference of percpu variable (var).
 */
#define put_cpu_var(var)                        \
do {                                    \
    (void)&(var);                           \
    preempt_enable();                       \
} while (0)

#define get_cpu_ptr(var)                        \
({                                  \
    preempt_disable();                      \
    this_cpu_ptr(var);                      \
})
```

The corresponding `put_cpu_var()` or `put_cpu_ptr()` re-enables preemption. Preemption pinning prevents task migration; it does not prevent a hardirq on the same CPU from touching the same per-CPU object. If task and hardirq both update it, use an operation/API whose interrupt semantics fit or exclude the interrupt. Remote CPUs can also access another CPU's instance with explicit APIs, so “per-CPU” is a locality layout, not universal ownership enforcement.

Per-CPU state is inappropriate when every update must immediately produce one globally linearizable value, when aggregation cost dominates, or when cross-CPU access is the common path. Approximate statistics are a natural fit; a one-copy ownership decision often is not.

Bounded lab: analogs and simulators only

Use the Day 5 exercises under `projects/linux-kernel-sync`:

- a userspace lifetime model with explicit get/put and exactly one destructor;
- a sequence-counter simulator whose reader copies two related fields and retries if the generation changes;
- a thread-local/per-worker counter benchmark that aggregates after bounded work and compares contention with one global atomic counter.

Bound thread counts, iterations, and elapsed time. Inject misuse only into a simulator that reports it safely; do not attempt overflow, use-after-free, or privileged kernel tracing on the host.

The exercises can prove their own destructor count, retry count, final aggregate, and timing sample. They cannot prove Linux `refcount_t` saturation, that a kernel seqcount branch executed, that a task stayed on one CPU, or that the host kernel used the pinned implementation. A thread-local analog is not Linux per-CPU storage. Print `kernel_path_claim=none` and record `uname -r` separately from source evidence.

Connections and limits

Week 11 supplied atomic ordering and the distinction between safety and progress. Week 14 supplied preemption, hardirq, and softirq context—the exact facts that govern current-CPU pointer stability and seqcount writer exclusion. Week 15 explains migration and why disabling preemption keeps a task on its current runqueue temporarily; it does not make aggregation globally atomic.

Use refcounts for lifetime, not field exclusion. Use seqcounts for short, retryable snapshots with externally serialized writers, not pointer lifetime. Use per-CPU storage for locality, not an instantaneous global truth. Combining them is common, but each invariant still needs a separate argument.

The excerpts establish only the named Linux 6.12 commit. RCU integration and later upstream behavior are outside this day's proof and are **potentially changed** until checked against named revisions.

Completion check

For one imagined statistics object, write four separate statements:

```
lifetime owner → get/put rule → final release
field mutation → exclusion rule
snapshot reader → begin/copy/retry rule
hot update → per-CPU pin/update/aggregate rule
```

Then name one case where each mechanism is inappropriate. Label every assertion `Linux 0.12 historical`, `ToyOS actual`, `Linux 6.12 pinned`, `userspace analog`, or `not established`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Refcounts Seqlocks And Percpu State

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-005-refcounts-seqlocks-and-percpu-state.sh
```

- View lab source on GitHub

Day 6 — RCU Readers, Grace Periods, and Callbacks

Target time: approximately 3 hours

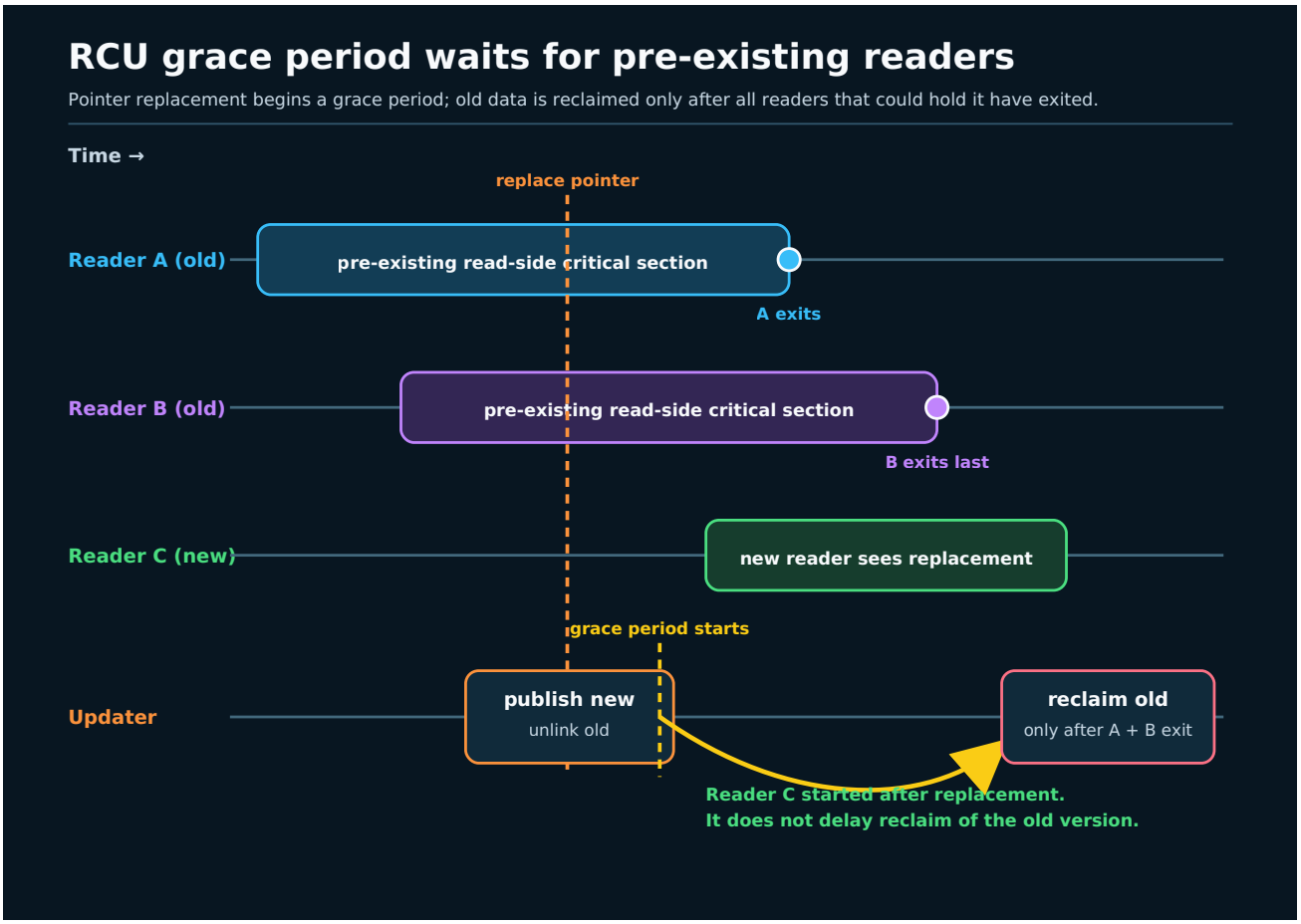
> Driving question: how can a reader follow a shared pointer without taking the updater's lock, while an updater replaces and eventually frees the old object?

The hard part is not making the pointer change. It is deciding when the object that used to be reachable can no longer be touched.

Imagine an interrupt-path lookup that runs thousands of times for every configuration update. A reader loads `current_policy` and examines the pointed-to object. An updater builds a replacement and publishes it. If the updater immediately frees the old object, a reader that loaded the old pointer one instruction earlier can dereference freed memory. If every reader instead takes one global lock, lifetime is easy but the read-mostly path pays cache-line bouncing, contention, and interrupt-context constraints.

RCU separates three events that a lock-centric explanation often collapses:

1. **publication** makes a fully initialized new object reachable;
2. **removal** makes an old object unreachable to future readers;
3. **reclamation** frees the old object only after every reader that could have seen it has finished.



1. Syntax: read the lifetime protocol before the implementation

The smallest conventional shape is:

```
struct policy __rcu *current_policy;

rcu_read_lock();
p = rcu_dereference(current_policy);
if (p)
    use(p->limit);
rcu_read_unlock();
```

`__rcu` is a sparse address-space annotation. It helps static checking; it is not a runtime reference count. `rcu_read_lock()` and `rcu_read_unlock()` delimit a read-side critical section. They do not mean “acquire and release one reader mutex,” and the reader normally does not update a shared reader count on every entry.

`rcu_dereference()` is the paired consuming operation for an RCU-published pointer. It performs the compiler and architecture-specific work needed to load that pointer and preserve dependency/ordering rules, while also feeding RCU debug checks. Plain `READ_ONCE()` is not a general substitute: it controls the shape of one access but does not express the RCU lifetime contract.

An updater usually looks like this:

```
mutex_lock(&policy_mutex);
old = rcu_dereference_protected(current_policy,
                lockdep_is_held(&policy_mutex));
rcu_assign_pointer(current_policy, new);
mutex_unlock(&policy_mutex);

synchronize_rcu();
kfree(old);
```

`rcu_assign_pointer()` publishes `new` with release ordering, so a reader that obtains it through the matching dereference cannot observe the pointer while still seeing pre-initialization contents. This ordering solves publication; it does not by itself solve reclamation.

The updater-side mutex remains essential when multiple updaters can race. RCU allows old and new *versions* to coexist for readers. It does not serialize two writers modifying a list, choosing replacements, maintaining counts, or enforcing a multi-field invariant.

Two callback-related pieces of syntax matter:

```
struct item {
    /* fields read under RCU */
    struct rcu_head rcu;
};

call_rcu(&old->rcu, free_item_rcu);
```

`container_of(head, struct item, rcu)` recovers the enclosing object in the callback. The embedded `rcu_head` is callback bookkeeping; it is not a reader reference count.

2. Linux 0.12: exclusion and lifetime before modern RCU

Linux 0.12 has no modern RCU API, no `rcu_read_lock()`, `rcu_dereference()`, `call_rcu()`, lockdep RCU validation, or Tree RCU grace period engine. It is also a uniprocessor kernel, so it does not face the modern problem of readers concurrently traversing shared structures on many CPUs under weak memory ordering.

Use `references/EARLY_LINUX_012_MAP.md` and Chapter 8.6 of the local commented Linux 0.12 PDF (`kernel/sched.c`, printed pages 351–372). The useful early model is coarse: kernel control flow excludes competing work through the UP execution context, interrupt disabling where interrupt concurrency matters, sleep/wakeup, and explicit object ownership or counts. Scheduling and interrupt return expose when another control flow can run. Object-specific code decides when a resource is no longer referenced and can be released.

That model teaches the right first question—“who could still hold this pointer?”—but not a modern answer. It has no per-CPU quiescent-state reporting, preemptible RCU readers, callback segmentation, CPU hotplug integration, NOHZ accounting, or SMP publication barriers. Saying “0.12 effectively used RCU because it was UP” would be wrong: absence of simultaneous CPU execution can simplify exclusion, but it is not the RCU API or grace-period protocol.

3. Why the mechanism evolved

Read-mostly kernel structures appear in routing, networking, credentials, process relationships, filesystems, scheduler topology, tracing, and device registries. Their readers may run in syscall, softirq, hardirq, or scheduler paths. A shared reader-writer lock can make every reader write the same lock cache line, and a writer can delay latency-sensitive readers. A reference count also writes shared state on acquisition and release, and it only helps if a reader can safely acquire a reference before the object disappears.

RCU changes the bargain:

- readers promise to keep each pointer use within a valid read-side interval unless they hand ownership to another lifetime mechanism;
- publishers promise to initialize before publishing with the correct primitive;
- removers prevent new readers from finding the old object;
- reclaimers wait for a grace period, or enqueue a callback that runs after it;
- writers still use locks or another update protocol to serialize updates.

A **grace period** is not “wait N milliseconds.” It is the interval needed to establish that all read-side critical sections that existed before the grace period began have ended. A **quiescent state** is state from which a CPU/task cannot still be in one of those pre-existing read-side sections for the RCU flavor in question. Context switches, user mode, idle, and explicit reporting participate according to flavor and configuration; none is a universal standalone definition.

Scheduling is therefore part of lifetime. On preemptible RCU, a task may be preempted while inside a read-side critical section and remain a blocker until it exits. Interrupt and softirq context also affect what counts as an active reader. NOHZ and idle CPUs require explicit accounting because “no tick” must not be mistaken for “no reader.”

4. Exact ToyOS boundary

The actual `projects/toyos/src/tasks.c` contains two ring-0 kernel tasks sharing one address space. `schedule()` alternates between them, updates `TASK_RUNNABLE`/`TASK_RUNNING`, and calls `context_switch`; `scheduler_tick()` can invoke it from the PIT interrupt path. `task_finish()` marks a task blocked. `projects/toyos/src/arch.c` increments `ticks`, acknowledges the PIC, and calls the scheduler for timer vector 32.

ToyOS does **not** implement RCU. It has one CPU, no `__rcu` annotations, read-side API, publication primitive, updater protocol, per-CPU RCU state, grace-period sequence, quiescent-state reporting, callback queue, CPU hotplug, NOHZ, preemptible reader tracking, lockdep, or sparse integration. Its `volatile bool task_done[2]` is not an RCU-protected object, and `volatile` does not create an SMP publication/lifetime protocol.

The exact bridge is narrower: ToyOS shows that scheduling and timer interrupts create control-flow boundaries and that an object can outlive the code that first exposed it. It cannot demonstrate that all pre-existing readers have passed a grace period. Adding a userspace epoch counter to a course lab does not retroactively add RCU to ToyOS.

5. Linux 6.12 pinned source

All source claims and excerpts below refer to commit `adc218676eef25575469234709c2d87185ca223a` in `/home/jd/Documents/linux-6.12`. Each excerpt is verbatim, contiguous, and bounded to 6–25 lines.

Publication is a release operation

`include/linux/rcupdate.h`, `rcu_assign_pointer`, lines 594–603:

```
#define rcu_assign_pointer(p, v)                              \
do {                                                  \
    uintptr_t _r_a_p__v = (uintptr_t)(v);                     \
    rcu_check_sparse(p, __rcu);                           \
                                                  \
    if (__builtin_constant_p(v) && (_r_a_p__v) == (uintptr_t)NULL)        \
        WRITE_ONCE((p), (typeof(p))(_r_a_p__v));              \
    else                                              \
        smp_store_release(&p, RCU_INITIALIZER((typeof(p))_r_a_p__v)); \
} while (0)
```

The non-NULL path is a release store. Initialize first, publish second. The special NULL case does not need to publish pointee initialization.

The reader-facing check couples pointer loading to the read-side condition:

`include/linux/rcupdate.h`, `rcu_dereference`, lines 753–758:

```
/**
 * rcu_dereference() - fetch RCU-protected pointer for dereferencing
 * @p: The pointer to read, prior to dereferencing
 *
 * This is a simple wrapper around rcu_dereference_check().
 */
#define rcu_dereference(p) rcu_dereference_check(p, 0)
```

The wrapper routes the ordinary reader through the checked dereference path. It does not prove sparse or `CONFIG_PROVE_RCU` was enabled on a host.

A quiescent state is reported into a protocol

`kernel/rcu/tree.c`, `rcu_momentary_eqs`, lines 360–379:

```
 * Let the RCU core know that this CPU has gone through the scheduler,
 * which is a quiescent state.  This is called when the need for a
 * quiescent state is urgent, so we burn an atomic operation and full
 * memory barriers to let the RCU core know about it, regardless of what
 * this CPU might (or might not) do in the near future.
 *
 * We inform the RCU core by emulating a zero-duration dyntick-idle period.
 *
 * The caller must have disabled interrupts and must not be idle.
 */
notrace void rcu_momentary_eqs(void)
{
    int seq;

    raw_cpu_write(rcu_data.rcu_need_heavy_qs, false);
    seq = ct_state_inc(2 * CT_RCU_WATCHING);
    /* It is illegal to call this from idle state. */
    WARN_ON_ONCE(!(seq & CT_RCU_WATCHING));
    rcu_preempt_deferred_qs(current);
}
```

The scheduler boundary matters, but this specialized function has caller preconditions. It must not be generalized into “every context switch immediately completes every grace period.”

Blocking wait and deferred callback are different updater choices

`kernel/rcu/tree.c`, `synchronize_rcu`, lines 4084–4098:

```
void synchronize_rcu(void)
{
    unsigned long flags;
    struct rcu_node *rnp;

    RCU_LOCKDEP_WARN(lock_is_held(&rcu_bh_lock_map) ||
             lock_is_held(&rcu_lock_map) ||
             lock_is_held(&rcu_sched_lock_map),
             "Illegal synchronize_rcu() in RCU read-side critical section");
    if (!rcu_blocking_is_gp()) {
        if (rcu_gp_is_expedited())
            synchronize_rcu_expedited();
        else
            synchronize_rcu_normal();
        return;
```

`synchronize_rcu()` waits for the required grace period and therefore belongs in sleepable process context. Calling it from a vanilla read-side critical section would wait for a set that includes the caller: the warning identifies that impossible lifetime dependency.

By contrast, the `call_rcu()` contract explicitly allows later readers to overlap the callback:

`kernel/rcu/tree.c`, `call_rcu` documentation, lines 3150–3164:

```
 * @head: structure to be used for queueing the RCU updates.
 * @func: actual callback function to be invoked after the grace period
 *
 * The callback function will be invoked some time after a full grace
 * period elapses, in other words after all pre-existing RCU read-side
 * critical sections have completed.  However, the callback function
 * might well execute concurrently with RCU read-side critical sections
 * that started after call_rcu() was invoked.
 *
 * RCU read-side critical sections are delimited by rcu_read_lock()
 * and rcu_read_unlock(), and may be nested.  In addition, but only in
 * v5.0 and later, regions of code across which interrupts, preemption,
 * or softirqs have been disabled also serve as RCU read-side critical
 * sections.  This includes hardware interrupt handlers, softirq handlers,
```

The callback is safe for the removed object because later readers cannot newly obtain that object after correct removal. It is not safe because “there are no readers anywhere.”

The callback is recorded and placed on per-CPU callback machinery:

`kernel/rcu/tree.c`, `__call_rcu_common`, lines 3084–3108:

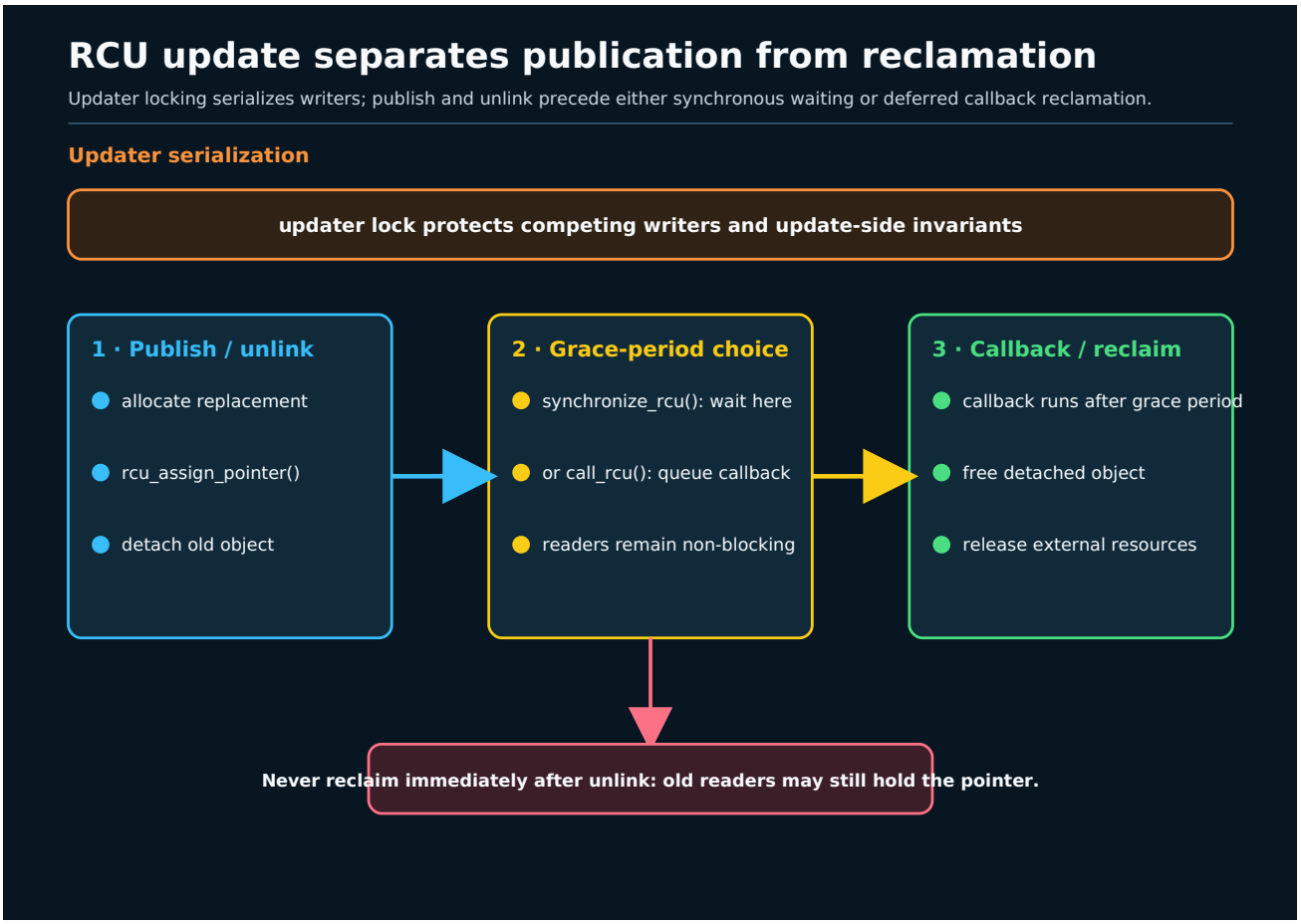
```
    head->func = func;
    head->next = NULL;
    kasan_record_aux_stack_noalloc(head);
    local_irq_save(flags);
    rdp = this_cpu_ptr(&rcu_data);
    lazy = lazy_in && !rcu_async_should_hurry();

    /* Add the callback to our list. */
    if (unlikely(!rcu_segcblist_is_enabled(&rdp->cblist))) {
        // This can trigger due to call_rcu() from offline CPU:
        WARN_ON_ONCE(rcu_scheduler_active != RCU_SCHEDULER_INACTIVE);
        WARN_ON_ONCE(!rcu_is_watching());
        // Very early boot, before rcu_init().  Initialize if needed
        // and then drop through to queue the callback.
        if (rcu_segcblist_empty(&rdp->cblist))
            rcu_segcblist_init(&rdp->cblist);
    }

    check_cb_ovld(rdp);

    if (unlikely(rcu_rdp_is_offloaded(rdp)))
        call_rcu_nocb(rdp, head, func, flags, lazy);
    else
        call_rcu_core(rdp, head, func, flags);
    local_irq_restore(flags);
```

This is asynchronous reclamation infrastructure, not immediate function invocation. The `local_irq_save()` section also connects callback enqueueing to interrupt concurrency; it does not turn the callback itself into hardirq work.



6. One lifetime example, end to end

Suppose a device lookup table contains `struct endpoint` objects.

1. A reader enters `rcu_read_lock()`, obtains an endpoint with `rcu_dereference()`, and reads fields.
2. The updater takes `endpoint_mutex`, unlinks the endpoint, and releases the mutex. Future lookups can no longer obtain it.
3. A reader that loaded the pointer before unlink may still be using it.
4. The updater either calls `synchronize_rcu()` and then frees it, or calls `call_rcu()` and lets the callback free it later.

The reader must not return the raw pointer and dereference it after `rcu_read_unlock()`. If the object must escape, acquire a stable reference *while still protected*, using an object-specific "increment unless zero" protocol whose removal path prevents resurrection. RCU protects the window in which that reference is acquired; the reference count protects the later lifetime. A refcount alone cannot close the load-versus-free race, and RCU alone does not grant indefinite ownership.

Sleeping is flavor-specific. Do not write the blanket rule "RCU readers can never sleep." In ordinary non-preemptible RCU, blocking is illegal. Preemptible RCU may preempt a reader but that does not make arbitrary blocking APIs valid. SRCU read-side sections may sleep and have a different API and accounting. Tasks RCU and Tasks Trace RCU wait on task-oriented

conditions for specialized users. RCU-bh and RCU-sched names remain relevant to API/history and validation, while vanilla RCU grace periods in this pinned era account for additional disabled-context regions. This lesson gives the map, not interchangeable recipes.

7. Bounded userspace epoch lab

The actual `projects/linux-kernel-sync/src/epoch_rcu_lab.c` is a userspace analogy, not a kernel RCU test. It starts exactly three pthread readers. Each loads a global atomic epoch with acquire ordering, stores that epoch in its own `active[]` slot, reads an atomic published integer, then clears its activity slot. The updater increments the epoch, release-publishes value 2, joins all three readers, and rejects any activity slot that still equals the retired epoch. A five-second alarm bounds failure.

Run the complete deterministic suite with:

```
make -C projects/linux-kernel-sync test-week16
```

For this exercise, the relevant literal success lines are `model=userspace_epoch_not_linux_rcu ... grace_condition=met`, `PASS epoch_rcu_lab`, and the suite's final `PASS test-week16`. Predict which reader timings can observe the old epoch before running it.

The model demonstrates explicit reader activity records and a condition checked before reuse. It has no removed heap object, free operation, callback mode, Tree RCU quiescent-state detection, interrupt semantics, CPU hotplug handling, callback offload, sparse annotation, or host-kernel barrier proof. Joining the threads is stronger and more direct than kernel RCU's normal read-side tracking. Treat the printed “grace condition” as the program's own predicate, not a Linux grace period.

`make -C projects/linux-kernel-sync tools` only reports whether `perf`, `trace-cmd`, `bpftrace`, and `lockstat` are present. It deliberately prints `SKIP` even when a tool exists because no optional tracing is run automatically. Preserve those SKIPs; do not synthesize events.

8. Evidence and limits

Keep these labels separate:

- **Linux 6.12 pinned-source proof:** the quoted code exists in the named path, function, and lines at the pinned commit.
- **Host observation:** literal output from an explicitly run bounded lab or interface, including host kernel and configuration facts actually collected.
- **Userspace analogy:** epoch bookkeeping illustrates lifetime ordering but does not execute kernel RCU.
- **Unknown:** which RCU configuration/flavor the host uses, whether a particular fast path ran, and callback latency if no trace was captured.
- **SKIP:** unavailable source project, compiler feature, tracing interface, or permission, with no inference of pass or failure.

Grace periods are not latency deadlines. A stalled CPU, long read-side section, starved grace-period kthread, interrupt-disabled loop, overload, or callback backlog can delay completion. `call_rcu()` avoids blocking the updater but can increase memory awaiting reclamation. `synchronize_rcu()` provides a simple sequencing point but can impose unacceptable latency and cannot be used in atomic context.

Finish by being able to narrate:

`initialize → release-publish → read-side acquire/dereference → unlink under update serialization → pre-existing readers finish → grace period completes → free directly or invoke callback`.

For every arrow, say whether it is an API contract, pinned-source structure, userspace-lab observation, or an untested hypothesis. That discipline is the difference between understanding RCU's lifetime mechanism and merely recognizing its function names.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Rcu Readers Grace Periods And Callbacks

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-006-rcu-readers-grace-periods-and-callbacks.sh
```

- View lab source on GitHub

Day 7 — Lockdep, KCSAN, RCU Stalls, and the Debug Report

Target time: approximately 3 hours

> Driving question: when concurrent kernel code fails only under an unlucky interleaving, what evidence distinguishes a bad lock order, illegal sleep, data race, lifetime error, seqcount misuse, or delayed RCU grace period?

Start with the symptom, not the favorite tool. “The machine hung” can mean two tasks formed a lock cycle, one CPU spun with interrupts disabled, an RCU grace period could not observe a quiescent state, or memory corruption damaged the state later inspected by the watchdog. “KCSAN printed a race” is narrower, but still does not tell you the intended invariant or the correct repair.

A useful report explains a causal boundary:

`first bad state → mechanism that permits it → evidence that observed it → small mitigation → regression that would catch its return`.

It also preserves negative evidence. If a required kernel configuration, tracefs access, symbol, or tool is unavailable, write `SKIP`. Tool absence is not a clean run, and source code containing a detector is not proof that the host enabled or triggered it.

Symptom-to-evidence tool map

Each diagnostic supports a bounded claim and has a specific class of claims it cannot establish.

Symptom	Best first tool	What it supports	What it cannot prove
lock cycle	lockdep	dependency context misuse	actual hang or hold time
suspected race	KCSAN	sampled conflicting access	absence means race-free
grace-period delay	RCU stall	insufficient progress threshold	single root-cause line
mechanism question	pinned source	implemented control flow	runtime branch occurred
process baseline	/proc	state cumulative counters	event ordering / causality

Triangulate: symptom + runtime evidence + pinned mechanism + explicit limits.

1. Syntax: recognize what a report can name

Lockdep reasons about **lock classes** and observed acquisition dependencies, not merely lock object addresses. If code acquires class B while holding class A, it can add an `A → B` edge. A later path that requires `B → A` closes a cycle. `lockdep_assert_held(&lock)` documents and checks a calling condition; it does not acquire the lock.

Context annotations carry different claims:

- `might_sleep()` asks the debugging machinery to warn if the current context cannot schedule safely;
- `in_interrupt()`, preemption count, IRQ state, and held spinlocks help explain why a blocking operation is illegal;
- `READ_ONCE()` / `WRITE_ONCE()` constrain individual compiler accesses but do not create a complete synchronization protocol;
- `data_race(expr)` marks an intentionally racy access for tooling and review; it does not make an unsafe race correct;
- `__rcu` plus `rcu_dereference()` expresses an RCU-protected pointer contract;
- `refcount_inc_not_zero()` is useful only inside a window in which loading the pointer itself is safe.

A sequence-counter reader has a retry loop:

```
do {
    seq = read_seqcount_begin(&state_seq);
    a = state.a;
    b = state.b;
} while (read_seqcount_retry(&state_seq, seq));
```

The fields from a failed iteration must be discarded. Writers must be serialized and non-preemptible for the reader population. A seqcount does not protect pointer lifetime: a reader can follow a pointer to freed memory before it discovers that a retry was needed.

KCSAN uses sampled watchpoints and delays to expose conflicting accesses. That makes it powerful and inherently incomplete: one run with no report does not prove race freedom. A KCSAN report is evidence of the displayed conflict under that build/run, not proof that every surrounding access lacks synchronization.

2. Linux 0.12: what is absent, and what remains useful

Linux 0.12 has no lockdep dependency graph, KCSAN instrumentation, sparse `__rcu` checking, `CONFIG_PROVE_RCU`, modern refcount saturation diagnostics, seqcount API, Tree RCU stall detector, ftrace, or `perf`.

Use `references/EARLY_LINUX_012_MAP.md` and Chapter 8.6 of the local commented Linux 0.12 PDF (`kernel/sched.c`, printed pages 351–372). Its uniprocessor kernel makes concurrency boundaries visible through interrupt enable/disable, task state, sleep/wakeup, and scheduling. Coarse UP exclusion and object-specific ownership/counting reduce the SMP interleavings modern tools must diagnose.

That is a historical baseline, not modern evidence. A UP kernel can still have interrupt/process-context races, illegal sleep, missed wakeups, and lifetime bugs. It simply lacks simultaneous execution on several CPUs and the current detectors. Do not project modern report strings or semantics backward onto it.

3. Why debugging had to evolve

SMP turns an implicit execution assumption into an observable ordering problem. Per-CPU data reduces contention but adds migration and cross-CPU publication. Preemption allows a task to stop while holding logical state. Hardirq, softirq, NMI, process context, and workqueues have different sleepability and nesting rules. Weakly ordered architectures expose missing barriers that x86-64 may hide. RCU intentionally permits old and new versions to coexist, so “pointer was removed” no longer means “object can be freed.”

No one detector spans all of that:

- **lockdep** validates modeled lock ordering, IRQ usage, and lock-context relationships on paths the instrumented kernel executes;
- **atomic-sleep checks** identify scheduling operations in a context whose preemption/IRQ/lock state forbids them;
- **KCSAN** samples conflicting memory accesses and can model selected weak memory effects, but it does not exhaust all interleavings;
- **sparse and `CONFIG_PROVE_RCU`** check RCU annotation and runtime read-side/lock conditions;
- **RCU stall warnings** report grace-period progress trouble, not one unique root cause;
- **KASAN/refcount diagnostics** can expose lifetime damage or misuse, but a refcount warning and a use-after-free are different events;
- **ftrace/perf** provide ordered activity or samples when configured and permitted; they do not establish an invariant by themselves.

4. Exact ToyOS boundary

The actual ToyOS has two ring-0 kernel tasks on one CPU. `projects/toyos/src/tasks.c` changes task state and switches contexts; `projects/toyos/src/arch.c` handles PIT vector 32, increments `ticks`, acknowledges the PIC, and invokes `scheduler_tick()`. Assertions and serial markers can prove only their explicit bounded self-tests.

ToyOS has no spinlock/mutex implementation, lock classes, dependency graph, SMP race, C11 atomic protocol, seqcount, refcount API, RCU, grace-period thread, stall detector, sparse build, KCSAN/KASAN instrumentation, ftrace, perf events, `/proc`, debugfs, or tracefs. Its interrupt path can teach that timer context interrupts task context and that sleeping rules depend on context. It cannot generate any of the Linux reports in this lesson.

Do not label a ToyOS assertion as lockdep, a serial timestamp as ftrace, or its single-CPU scheduling as an RCU grace period. Those substitutions erase the mechanism being studied.

5. Linux 6.12 pinned source: what the detectors actually report

All excerpts below are verbatim, contiguous, 6–25 lines, and taken from commit `adc218676eef25575469234709c2d87185ca223a` in `/home/jd/Documents/linux-6.12`.

Lockdep reports the cycle and both sides

`kernel/locking/lockdep.c`, `print_circular_bug_header`, lines 1947–1969:

```
print_circular_bug_header(struct lock_list *entry, unsigned int depth,
            struct held_lock *check_src,
            struct held_lock *check_tgt)
{
    struct task_struct *curr = current;

    if (debug_locks_silent)
        return;

    pr_warn("\n");
    pr_warn("======================================================\n");
    pr_warn("WARNING: possible circular locking dependency detected\n");
    print_kernel_ident();
    pr_warn("------------------------------------------------------\n");
    pr_warn("%s/%d is trying to acquire lock:\n",
        curr->comm, task_pid_nr(curr));
    print_lock(check_src);

    pr_warn("\nbut task is already holding lock:\n");

    print_lock(check_tgt);
    pr_warn("\nwhich lock already depends on the new lock.\n\n");
    pr_warn("\nthe existing dependency chain (in reverse order) is:\n");
```

“Possible” is precise. Lockdep found a dependency cycle in its modeled graph; the tasks need not have deadlocked in that run. The printed acquisition and held-lock stacks are the evidence needed to reconstruct the order.

Atomic-sleep checking reaches the reschedule validator

`kernel/sched/core.c`, `__might_sleep`, lines 8571–8586:

```
void __might_sleep(const char *file, int line)
{
    unsigned int state = get_current_state();
    /*
     * Blocking primitives will set (and therefore destroy) current->state,
     * since we will exit with TASK_RUNNING make sure we enter with it,
     * otherwise we will destroy state.
     */
    WARN_ONCE(state != TASK_RUNNING && current->task_state_change,
            "do not call blocking ops when !TASK_RUNNING; "
            "state=%x set at [<%p>] %pS\n", state,
            (void *)current->task_state_change,
            (void *)current->task_state_change);

    __might_resched(file, line, 0);
}
```

The eventual warning must be read with the stack and context state. The mere presence of `might_sleep()` in source does not show a host violated the rule.

KCSAN deliberately filters and serializes reporting

`kernel/kcsan/report.c`, `kcsan_report_known_origin`, lines 674–697:

```
    kcsan_disable_current();
    /*
     * Because we may generate reports when we're in scheduler code, the use
     * of printk() could deadlock. Until such time that all printing code
     * called in print_report() is scheduler-safe, accept the risk, and just
     * get our message out. As such, also disable lockdep to hide the
     * warning, and avoid disabling lockdep for the rest of the kernel.
     */
    lockdep_off();

    if (!prepare_report_consumer(&flags, &ai, other_info))
        goto out;
    /*
     * Never report if value_change is FALSE, only when it is
     * either TRUE or MAYBE. In case of MAYBE, further filtering may
     * be done once we know the full stack trace in print_report().
     */
    if (value_change != KCSAN_VALUE_CHANGE_FALSE)
        print_report(value_change, &ai, other_info, old, new, mask);

    release_report(&flags, other_info);
out:
    lockdep_on();
    kcsan_enable_current();
```

This code itself states an evidence limit: reporting can arise in scheduler code, printing has its own locking risk, and candidate reports can be filtered. No report is not proof that no conflict occurred.

Seqcount writer rules are part of correctness

`include/linux/seqlock.h`, `write_seqcount_begin`, lines 459–480:

```
 * write_seqcount_begin() - start a seqcount_t write side critical section
 * @s: Pointer to seqcount_t or any of the seqcount_LOCKNAME_t variants
 *
 * Context: sequence counter write side sections must be serialized and
 * non-preemptible. Preemption will be automatically disabled if and
 * only if the seqcount write serialization lock is associated, and
 * preemptible.  If readers can be invoked from hardirq or softirq
 * context, interrupts or bottom halves must be respectively disabled.
 */
#define write_seqcount_begin(s)                     \
do {                                        \
    seqprop_assert(s);                      \
                                    \
    if (seqprop_preemptible(s))                 \
        preempt_disable();                  \
                                    \
    do_write_seqcount_begin(seqprop_ptr(s));            \
} while (0)

static inline void do_write_seqcount_begin(seqcount_t *s)
{
    do_write_seqcount_begin_nested(s, 0);
```

A report about inconsistent fields should therefore ask about writer serialization, preemption, reader interrupt context, retry handling, and pointer lifetime—not merely whether the counter changed.

PROVE_RCU gives a runtime suspicious-use report

`kernel/locking/lockdep.c`, `lockdep_rcu_suspicious`, lines 6776–6796:

```
void lockdep_rcu_suspicious(const char *file, const int line, const char *s)
{
    struct task_struct *curr = current;
    int dl = READ_ONCE(debug_locks);
    bool rcu = warn_rcu_enter();

    /* Note: the following can be executed concurrently, so be careful. */
    nbcon_cpu_emergency_enter();
    pr_warn("\n");
    pr_warn("=============================\n");
    pr_warn("WARNING: suspicious RCU usage\n");
    print_kernel_ident();
    pr_warn("-----------------------------\n");
    pr_warn("%s:%d %s!\n", file, line, s);
    pr_warn("\nother info that might help us debug this:\n\n");
    pr_warn("\n%srcu_scheduler_active = %d, debug_locks = %d\n%s",
           !rcu_lockdep_current_cpu_online()
            ? "RCU used illegally from offline CPU!\n"
            : "",
           rcu_scheduler_active, dl,
           dl ? "" : "Possible false positive due to lockdep disabling via debug_locks = 0\n");
```

The last line matters: even the detector reports when its own lock-debug state weakens confidence. Sparse is complementary compile-time evidence; it can flag annotation misuse without observing execution.

The RCU stall report carries progress context

`kernel/rcu/tree_stall.h`, `print_cpu_stall_info`, lines 482–506:

```
static void print_cpu_stall_info(int cpu)
{
    unsigned long delta;
    bool falsepositive;
    struct rcu_data *rdp = per_cpu_ptr(&rcu_data, cpu);
    char *ticks_title;
    unsigned long ticks_value;
    bool rcuc_starved;
    unsigned long j;
    char buf[32];

    /*
     * We could be printing a lot while holding a spinlock.  Avoid
     * triggering hard lockup.
     */
    touch_nmi_watchdog();

    ticks_value = rcu_seq_ctr(rcu_state.gp_seq - rdp->gp_seq);
    if (ticks_value) {
        ticks_title = "GPs behind";
    } else {
        ticks_title = "ticks this GP";
        ticks_value = rdp->ticks_this_gp;
    }
    delta = rcu_seq_ctr(rdp->mynode->gp_seq - rdp->rcu_iw_gp_seq);
```

An RCU stall warning means grace-period progress exceeded the detector's threshold. It can result from a long read-side section, a CPU looping with interrupts/preemption disabled, a starved RCU kthread, timer/clock trouble, overload, virtualization delay, or detector/console effects. The warning is a starting point, not a verdict of “RCU bug.”

6. Six symptom-driven reports

Each report below has the same spine: symptom, competing causes, evidence, mitigation, regression, and limit.

A. Deadlock or lock inversion

Symptom: two operations stop making progress; blocked stacks show each waiting near a different lock, or lockdep prints a circular dependency.

Competing causes: an actual `A ↔ B` wait cycle; a lockdep-predicted cycle not realized in this run; a long lock holder; missed wakeup; or unrelated CPU starvation.

Evidence: preserve the complete lockdep report, kernel release/config, both acquisition stacks, held locks, lock class names, PID/CPU, and surrounding timestamps. If lockdep is unavailable, collect task stacks and owner/waiter state where safely exposed, but label cycle reconstruction as a hypothesis. ftrace lock events may establish order only if configured before the incident.

Mitigation: impose one documented global order, split the critical section, or move blocking work outside the lock. Do not silence lockdep by changing annotations unless the modeled class relationship is genuinely wrong.

Regression: a bounded stress test that executes both orderings and has a watchdog, deterministic completion count, and lockdep enabled. A pass reduces risk; it does not prove all paths cycle-free.

B. Sleeping in atomic context

Symptom: `BUG: sleeping function called from invalid context`, an atomic- sleep warning, or latency/hang near an allocation or mutex from IRQ/spinlocked code.

Competing causes: blocking under a spinlock; blocking with IRQs or preemption disabled; wrong allocation flags; a misleading secondary stack after corruption.

Evidence: retain the first warning, preemption/IRQ state, held-lock list, call stack, and allocation/sleep site. Source inspection establishes whether the API may sleep; only the runtime report establishes the observed context.

Mitigation: move work to a sleepable context, use a truly atomic operation with bounded failure handling, or redesign ownership. Replacing every `GFP_KERNEL` with `GFP_ATOMIC` can hide architecture problems and exhaust reserves.

Regression: execute the interrupt/softirq/process-context variants under `CONFIG_DEBUG_ATOMIC_SLEEP`; require completion and absence of the specific first warning, while reporting configuration explicitly.

C. Data race

Symptom: KCSAN reports two conflicting accesses, or state intermittently contains a combination no serialized execution permits.

Competing causes: missing lock/barrier; intentionally approximate statistics; stale publication; lifetime reuse making two unrelated objects share an address; or a false causal attribution from a later symptom.

Evidence: preserve both KCSAN stacks, access sizes/types, address/symbol, value-change information, build config, and reproducer. Map both accesses to the intended synchronization invariant. `READ_ONCE()` can prevent tearing or compiler merging for supported sizes, but does not automatically fix a multi-field invariant.

Mitigation: use one ownership/locking protocol, a correctly paired release/acquire publication, an atomic RMW, or an explicitly documented benign race only when the semantics permit it.

Regression: repeated bounded runs on the instrumented kernel plus a deterministic invariant check. KCSAN silence remains sampled negative evidence.

D. Refcount use-after-free

Symptom: KASAN reports access to freed memory; refcount diagnostics report underflow/saturation; or a crash follows object removal.

Competing causes: missing reference; double put; acquiring a reference from an already stale pointer; RCU removal followed by premature free; or unrelated heap corruption.

Evidence: preserve the first allocator/free/access stacks, object address, refcount warning, and ownership transition. A later page fault alone cannot identify which put was wrong. Source proof of `refcount_inc()` does not prove the pointer was alive when the increment began.

Mitigation: define who owns each reference; use RCU or the containing lock to stabilize lookup; use increment-unless-zero where resurrection must fail; remove visibility before final put; defer reclamation when old readers remain.

Regression: force lookup/removal overlap in a bounded instrumented test, assert constructor/destructor balance, and check deterministic completion. KASAN unavailable means `SKIP KASAN`, not “no UAF.”

E. Seqcount misuse

Symptom: readers observe impossible field combinations, spin for excessive time, or lockdep/KCSAN identifies writer-side misuse.

Competing causes: writers are not serialized; writer is preempted while the sequence is odd; interrupt reader can interrupt a writer; reader forgot retry; reader used values from a failed iteration; or protected data includes a pointer whose target was freed.

Evidence: record writer lock/context, sequence values when safely instrumented, reader retry count, and whether IRQ/BH masking matches reader contexts. A perf sample in the retry loop shows CPU consumption, not why the writer failed to finish.

Mitigation: use `seqcount_LOCKNAME_t` or `seqlock_t` when appropriate, serialize writers, disable the required interrupt class, keep writes bounded, and give pointer targets a separate lifetime mechanism.

Regression: bounded writer/readers with invariant assertions, retry budget telemetry, and interrupt-context coverage where the real API permits it.

F. RCU stall

Symptom: the kernel prints an RCU CPU/task stall warning, callback backlog grows, or an updater waiting in `synchronize_rcu()` takes unexpectedly long.

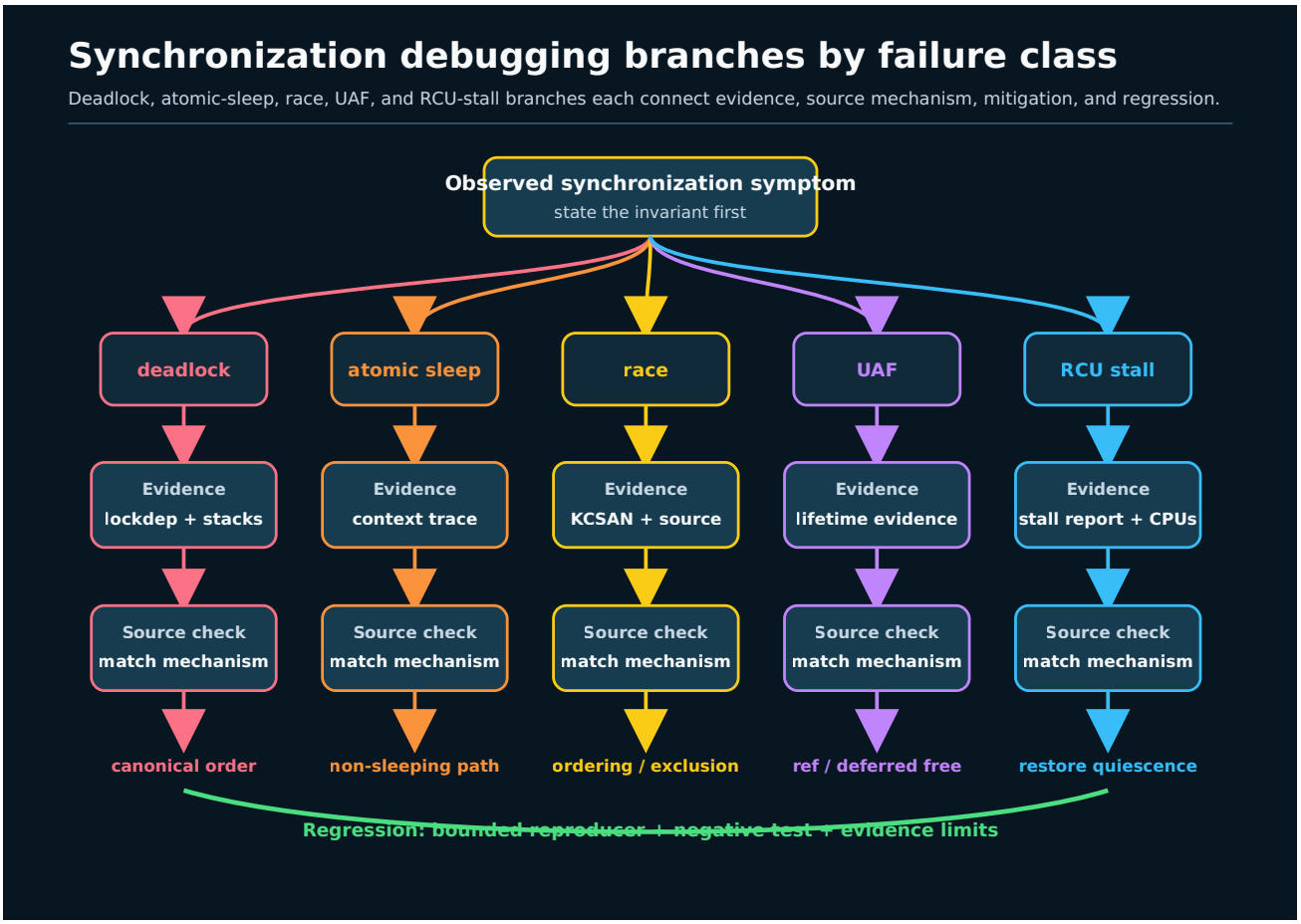
Competing causes: long reader; CPU not scheduling or not taking timer interrupts; excessive IRQ/preempt-disabled loop; starved grace-period kthread; console flood; guest vCPU descheduling; or genuine RCU accounting defect.

Evidence: retain the entire first stall splat, grace-period sequence, listed CPUs/tasks, `ticks this GP` / `GPs behind`, softirq and context-switch data, stacks, kernel command line, virtualization context, and wall-clock interval. Use ftrace events only when already enabled or safely reproduced; tracing after the stall cannot reconstruct lost history.

Mitigation: shorten the proven reader/atomic section, restore scheduling or interrupt progress, fix kthread starvation, reduce console disturbance, or repair host vCPU service. Raising the timeout only changes detection unless the event was a justified false positive.

Regression: repeat the bounded workload with callback-drain completion, grace-period progress markers, and the same load/configuration. Do not promise a universal maximum grace-period latency.

7. Evidence workflow and tool boundaries



Use this order:

1. Define the first externally visible symptom and the time/task/CPU boundary.
2. Preserve the first warning; later failures may be consequences.
3. Record exact kernel release, config evidence, boot parameters, taint, CPU topology, and bare-metal/guest status that are actually available.
4. Classify context: task, softirq, hardirq, NMI; IRQ/preemption state; held locks; RCU read-side state.
5. Write at least two causes with different predicted evidence.
6. Use the least invasive available detector or trace.
7. Map observed frames to pinned source while marking version mismatch.
8. Apply one reversible mitigation tied to the strongest evidence.
9. Run a bounded regression with deterministic completion plus detector output.

Tool boundaries:

- **lockdep:** executed lock dependencies and modeled contexts; not every unexecuted path and not arbitrary data-race detection.
- **KCSAN:** sampled conflicting accesses; not exhaustive proof and not object ownership intent.
- **sparse:** static annotation/address-space checking; not runtime branch execution.
- **PROVE_RCU:** runtime checking for instrumented RCU conditions; dependent on config, path execution, and lockdep state.

- **RCU stall detector:** delayed grace-period progress; not unique root cause.
- **ftrace:** ordered enabled events/functions within buffer and clock limits; dropped/disabled events remain unknown.
- **perf:** samples or configured counters/events; sampling absence is not execution absence, and unwinding may be incomplete.

If any tool or interface is missing, disabled, denied, or unsafe to enable, write `SKIP: <tool> — <reason>`. Do not mount tracefs, change lockdown, load a module, reboot into a debug kernel, or alter production controls merely to make the report look complete.

8. Bounded userspace report lab

The actual `projects/linux-kernel-sync/src/debug_report.c` performs a read-only self-observation. It opens `/proc/self/status` separately for each requested field and checks only `State`, `Threads`, `voluntary_ctxt_switches`, and `nonvoluntary_ctxt_switches`. It then prints:

```
lockdep=kernel-config-dependent
kcsan=kernel-config-and-runtime-dependent
rcu_stall=no_claim_from_userspace
```

Run the complete bounded suite with:

```
make -C projects/linux-kernel-sync test-week16
```

The suite builds seven strict-C17 pthread programs, runs a read-only checker against the pinned source commit, and uses bounded alarms where a scenario could otherwise hang. `epoch_rcu_lab` labels itself `model=userspace_epoch_not_linux_rcu`; `debug_report` ends with `PASS debug_report`. These are userspace completion and analogy evidence only. The `/proc` fields are snapshots or cumulative counters and prove no lock cycle, data race, refcount transition, seqcount retry, or RCU stall.

`make -C projects/linux-kernel-sync tools` checks `perf`, `trace-cmd`, `bpftrace`, and `lockstat`, but runs none of them. Every result is deliberately `SKIP`: either the tool is missing, or it is present but requires explicit learner action and suitable permission. The core lab does not inspect host lockdep, KCSAN, PROVE_RCU, KASAN, sparse, ftrace, or an RCU stall log. Preserve that boundary instead of converting source availability into host observation.

9. Final report format

Write the report in nine compact sections:

1. **Symptom:** first failure, affected operation, PID/task/CPU, and interval.
2. **Environment:** exact observed host facts and relevant `SKIP`s.
3. **Mechanism invariant:** lock order, context rule, happens-before edge, reference ownership, seqcount contract, or RCU lifetime condition.
4. **Competing hypotheses:** at least two, each with a distinguishing prediction.
5. **Host observation:** literal warnings, stacks, counters, or traces.
6. **Pinned-source proof:** path/function/line/commit and what structure it proves—never a claim that the host executed it.

7. **Causal assessment:** strongest explanation, contradictory evidence, unknown arrows, and version mismatch.
8. **Mitigation:** one reversible change tied to the mechanism.
9. **Regression and limits:** deterministic check, bounded stress, detector configuration, sampling/perturbation, and remaining alternatives.

Finish with four explicit labels:

- `Linux 6.12 pinned source:`
- `host observed:`
- `unknown:`
- `potentially changed:`

The quality of the report is not the number of tools named. It is whether a reviewer can tell which synchronization rule was violated, which evidence actually came from the host, why the mitigation addresses that rule, and what the regression still cannot prove.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Lockdep Kcsan Rcu Stalls And Debug Report

Format: shell lab

What you will do: set -eu

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 16-kernel-synchronization-memory-ordering-and-rcu/challenges/day-007-lockdep-kcsan-rcu-stalls-and-debug-report.sh
```

- View lab source on GitHub